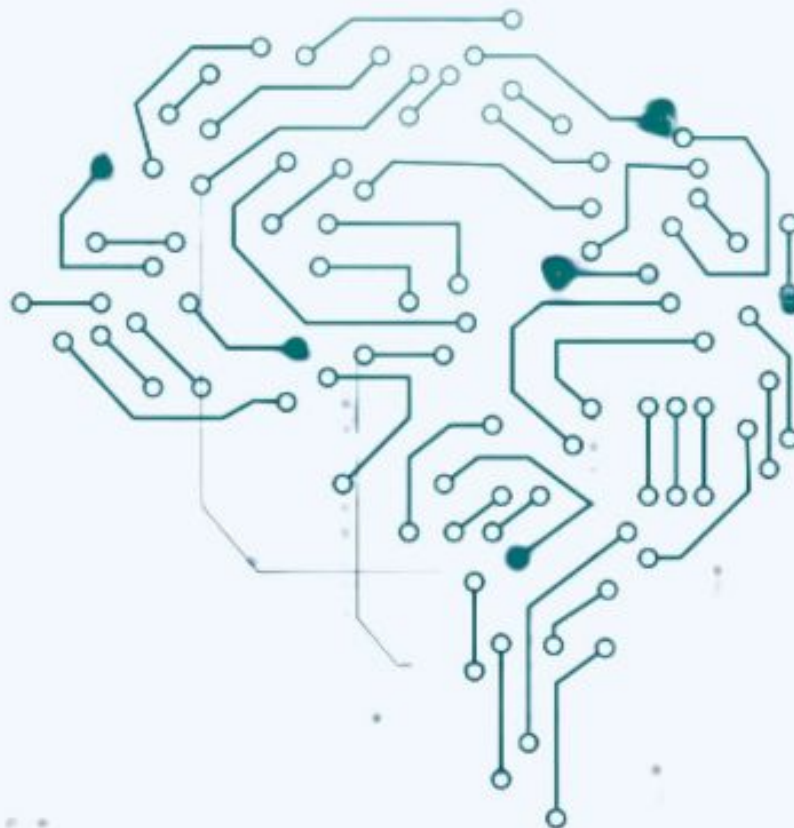


MACHINE LEARNING ALGORITHMS

SIMPLIFIED

Dr Lino A Tharakan



TAN PUBLISHING

MACHINE
LEARNING
ALGORITHMS
Simplified

Dr Lino A Tharakan

Dr Lino A Tharakan (Author)

With an illustrious academic journey spanning over 19 years, he presently serves as a distinguished faculty member in the domain of Electronics Science within the Department of Computer Applications at Mar Thoma Institute of Information Technology, affiliated with the esteemed University of Kerala.

His academic credentials boast a Master's degree in Electronics from Barkatullah University, followed by the attainment of a doctoral degree in Elec-

tronics from Bharathiar University, Coimbatore, in 2017.

His intellectual contributions extend beyond the classroom, encompassing the authorship of two books and the composition of two book chapters published by Springer and IGI Global.

His erudition is further evident in the plethora of research endeavors, featuring over 10 national and 13 international papers, all indexed in Scopus journals. Actively engaging in academic discourse, he has made impactful presentations at six national and international conferences, seminars, and symposia, including notable IEEE conferences.

Presently, his scholarly pursuits are centered around cutting-edge research in cryptographic data handling within the realm of IoT devices.

His multifaceted research interests encompass wireless sensor networks, data security, compression in sensor networks, and power management in WSN networks, showcasing a profound com-

mitment to advancing knowledge in these domains.

Dedication

With profound gratitude to Dr. Dhanasekharan,
my guiding light through the academic odyssey
of my Ph.D., whose wisdom and mentorship
have shaped the pages of this journey

MACHINE LEARNING ALGORITHMS

Simplified

Dr Lino A Tharakan

MACHINE LEARNING ALGORITHMS SIMPLIFIED

lino.miit@gmail.com

Cover Design : TAN Publishing

First Edition : December 2023

Publisher : Tan Publishing

Price: Rs. 110

PREFACE

Machine Learning Simplified: A Comprehensive Guide to Supervised, Unsupervised, and Semi-Supervised Algorithms

In today's data-driven world, machine learning has become an essential tool for businesses and organizations of all sizes. This powerful technology enables computers to learn from data without being explicitly programmed, making it possible to solve complex problems and uncover hidden insights. However, understanding the intricacies of machine learning can be daunting, especially for those without a background in computer science or mathematics.

Your Essential Guide to Machine Learning Fundamentals

Enter "Machine Learning Simplified," a comprehensive guide that breaks down the complex concepts of ma-

chine learning into easily understandable terms. This book is designed for anyone interested in learning about machine learning, whether they are complete beginners or experienced professionals seeking to expand their knowledge.

Demystifying Supervised Learning Algorithms

The book begins by introducing the fundamental concepts of supervised learning, where algorithms are trained on labeled data to learn a mapping from inputs to outputs. This section covers a wide range of supervised learning algorithms, including linear regression, logistic regression, decision trees, support vector machines, and neural networks.

Understanding Unsupervised Learning Techniques

Next, the book delves into the world of unsupervised learning, where algorithms are tasked with finding patterns or structure in unlabeled data. This section covers various unsupervised learning techniques, including clustering algorithms, dimensionality reduction techniques, and anomaly detection algorithms.

Bridging the Gap with Semi-Supervised Learning

The book then explores the concept of semi-supervised learning, which combines labeled and unlabeled data to improve the performance of machine learning algorithms. This section covers various semi-supervised learning techniques, including self-training, co-training, and graph-based methods.

Real-World Applications and Practical Examples

Throughout the book, the author provides numerous real-world examples to illustrate the concepts of machine learning. These examples demonstrate how machine learning is used in various domains, such as healthcare, finance, marketing, and social media.

A Comprehensive Resource for All Levels of Expertise

Whether you are a complete beginner or an experienced professional seeking to expand your knowledge of machine learning, "Machine Learning Simplified" is an essential resource. The book's clear and concise explanations, combined with its abundance of real-world examples, make it an ideal guide for anyone interested in learning about this powerful technology.

INDEX

Chapter 1 : Machine Learning Algorithms

- 1.1** Introduction
- 1.2** Supervised learning
- 1.3** Unsupervised learning
- 1.4** Semi-supervised learning
- 1.5** Reinforcement learning

Chapter 2 : Supervised Learning

2.1 Introduction

2.1.1. Classification Algorithms

2.1.2. Regression Algorithms

2.2 Linear Regression

2.2.1 Example to build a linear regression
model

2.2.2 A Python program with above equation

to predict the sales price of a given
House

2.2.3 Linear regression Applications in real-
world

2.3 Logistic regression

-
- 2.3.1 Python program that uses logistic regression to predict whether an email is spam or not based on its content

2.4 **Decision trees**

- 2.4.1 Example of how to create a decision tree using scikit-learn, a machine learning library in Python

- 2.4.2 Applications of Decision Tree model

2.5 **Random Forest Algorithm**

- 2.5.1 Example of building a Random Forest model in Python

- 2.5.2 Example of Random Forest Algorithm

2.6 **The Naive Bayes algorithm**

- 2.6.1 Example spam emails detection

2.7 **Support Vector Machines (SVMs)**

2.7.1 Example of how SVMs work for a binary

classification problem

2.7.2 Example of how an SVM classifier be used to classify spam emails

2.7.3 Python code example for training an SVN (Support Vector Machine) machine learning model on a simple classification problem

2.8 Neural networks

2.8.1 An example of a simple 3-layer neural network

2.8.2 An example of how you could create a neural network classification model using Python and the Keras library

2.9 K-Nearest Neighbors (KNN)

2.9.1 KNN Working

2.9.2 Algorithmic Steps

2.9.3 Example of classifying Iris flowers
using the K-Nearest Neighbors (KNN) al-
gorithm

Chapter 3 : Unsupervised Learning

3.1 Introduction

3.2 Clustering

3.2.1. K-means clustering

3.2.2 Hierarchical clustering

3.2.2.1 Example of hierarchical clustering

3.2.3 Agglomerative method

3.3.4 Divisive hierarchical clustering

3.3.4.2 Example for Divisive hierarchical clustering

3.3.4.3 Comparison

3.3.3 Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

3.3.3.1 Example to implements the DBSCAN algorithm

3.3.4 Gaussian Mixture Models (GMM)

3.3.5 Spectral clustering

3.3.5.1 Spectral clustering has the following steps:

3.4 Dimensionality reduction algorithms

3.4.1 Applications

3.4.2 Popular types of Dimensionality reduction algorithms

3.5 Principal Component Analysis (PCA)

3.5.1 Steps in Principal Component Analysis:

3.5.2 Example of Principal Component Analysis (PCA) using Python with the popular scikit-learn library

3.6.1 t-Distributed Stochastic Neighbor Embedding (t-SNE):

3.6.1 Example of a Python code using the t-SNE algorithm from the scikit-learn library.

3.6.2 Explanation of the code

3.7 Autoencoders

3.7.1 Architecture Setup

3.7.2 Example of a simple autoencoder implemented in Python using the Keras library

Chapter 4 : Semi-Supervised Learning

4.1 Introduction

4.2 The Self-Training Algorithm

4.3. Co-Training Algorithm

4.3.1 Example : Implementing Co-Training
for

Semi-Supervised Learning using Multiple Views of Data in PythonGraph-based learning

4.4 Multi-view learning

4.4.1 Example : Canonical Correlation

Analysis (CCA) using the scikit-learn library

4.5 Graph-based learning

4.6 Machine learning applications and Uses cases

CHAPTER 1

Machine learning algorithms

MACHINE LEARNING ALGORITHMS

1. Introduction

Machine learning algorithms are mathematical models or techniques that enable machines to learn from data and make predictions or decisions without being explicitly programmed. These algorithms are at the core of modern artificial intelligence (AI) and are used in a wide range of applications, such as image and speech recognition, natural language processing, recommendation systems, and predictive modeling.

Mathematical models are representations of real-world phenomena or systems using mathematical equations or formulas. They are used to describe and understand complex systems or processes, to make predictions or forecasts, and to test hypotheses or theories.

Mathematical models can take different forms, depending on the complexity of the system being modeled and the type of questions being asked. Some types of mathematical models are

- **Linear models:** These models use linear equations to describe the relationship between the input and output variables. They are often used in simple systems where the relationship between the variables is straightforward.
- **Non-linear models:** These models use non-linear equations to describe the relationship between the input and output variables. They are often used in complex systems

where the relationship between the variables is not straightforward.

- **Statistical models:** These models use statistical techniques to describe the relationship between the input and output variables. They are often used in situations where there is uncertainty or variation in the data.
- **Dynamic models:** These models take into account the changes that occur over time in a system. They use differential equations to describe the changes in the system over time.

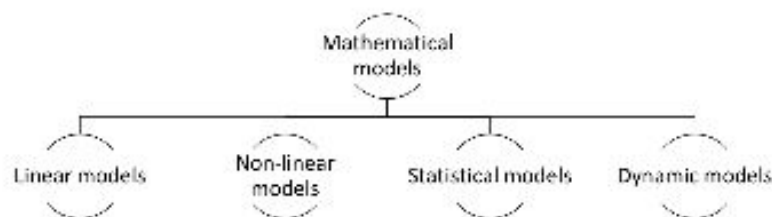


Figure 1.1: ML Mathematical Model Classification

There are several types of Machine learning algorithms, including:

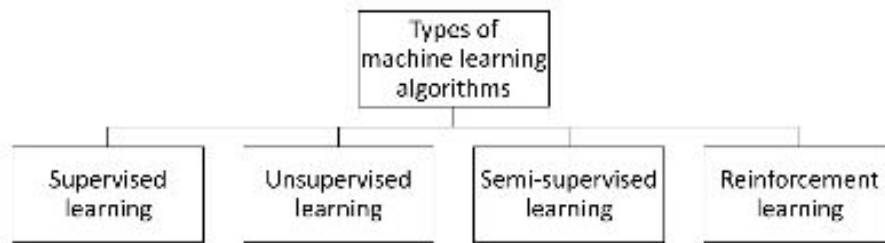


Figure 1.2 : Types of Machine Learning Algorithms

1. **Supervised learning:** In this type of learning, the algorithm is trained on labeled data, where the desired output is known for each input. The algorithm learns to map the input to the output by minimizing the error between its predictions and the actual output.
2. **Unsupervised learning:** In this type of learning, the algorithm is trained on unlabeled data, where there is no known output. The algorithm learns to identify patterns and structure in the data without any guidance.
3. **Semi-supervised learning:** This is a combination of supervised and unsupervised

learning, where the algorithm is trained on a mix of labeled and unlabeled data. The algorithm learns from the labeled data and generalizes to the unlabeled data.

4. Reinforcement learning: In this type of learning, the algorithm learns from feedback in the form of rewards or penalties. The algorithm tries to maximize the reward by taking actions that lead to the desired outcome.

Some common machine learning algorithms include linear regression, logistic regression, decision trees, random forests, k-nearest neighbors, support vector machines, and neural networks. The choice of algorithm depends on the problem domain, the size and complexity of the data, and the desired performance metrics.

1.2 Supervised learning

Supervised learning is a type of machine learning in which an algorithm is trained on labelled data, where the desired output is known for each input. The algorithm learns to map the input to the output by minimizing the error between its predictions and the actual output. This type of learning is often used for prediction or classification tasks, where the goal is to learn a mapping from input variables to output variables.



In machine learning, labeled data refers to data that has been pre-labeled or annotated with one or more output variables or "labels" that correspond to each input sample. The output variable or label is the desired output or target that the machine learning algorithm is trying to predict or classify.

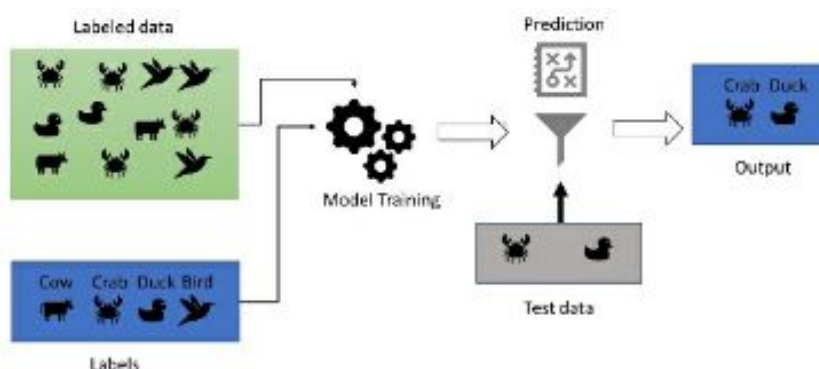


Figure 1.4.1: Concept of Supervised learning

Example of supervised learning:

Let's say we have a dataset of housing prices in a city. Each data point represents a house and has features such as the size, number of bedrooms, and location of the house. The desired output variable is the price of the house.



In machine learning and data science, a dataset is a collection of data that is organized and stored in a structured format for analysis and processing. A dataset typically consists of a set of data points or examples, each of which has one or more features or attributes that describe it.

The dataset contains many examples of houses and their corresponding prices, and we can use this dataset to train a supervised learning algorithm to predict the price of a new house based on its features. We can split the dataset into two parts: a training set and a test set.

During the training phase, the algorithm is presented with the training set, which includes the input features of each house and its corresponding price. The algorithm learns to map the input features to the output price by adjusting its internal parameters to minimize the difference between its predictions and the actual prices.



In machine learning, the training phase refers to the period during which a machine learning model is "trained" or "taught" using a labeled dataset. During the training phase, the machine learning model is exposed to a set of input data along with the corresponding output labels or target values, and it learns to recognize patterns in the input data that are associated with the output labels.

Once the algorithm is trained, we can use it to make predictions on new, unseen houses in the test set. We provide the input features of the new house to the algorithm, and it predicts the price of the house based on what it learned from the training data. We can then compare the predicted

prices to the actual prices to evaluate the performance of the algorithm.

In this example, the supervised learning algorithm learned to predict the price of a house based on its features. This type of learning can be applied to many other prediction or classification tasks, such as predicting the probability of a customer buying a product, classifying images into different categories, or predicting the risk of a disease based on patient data.

1.3 Unsupervised learning

Unsupervised learning is a type of machine learning in which the algorithm is trained on unlabeled data, which means that there are no pre-existing labels or output variables associated with the data. The goal of unsupervised learning is to identify patterns and relationships within the data that can be used to gain insights or make predictions.



A pattern is a regularity or recurring struc-

ture within the data that can be identified by a machine learning algorithm. Patterns can take many forms, such as clusters of similar data points, trends or correlations within the data, or frequently occurring sequences or motifs. Patterns and relationships refer to the underlying structure and correlations within a dataset. These patterns and relationships can often be identified using statistical methods and machine learning algorithms.

Unlike supervised learning, where the algorithm is trained on labeled data, unsupervised learning algorithms must learn to identify patterns and relationships within the data without any guidance or feedback from labeled output data.

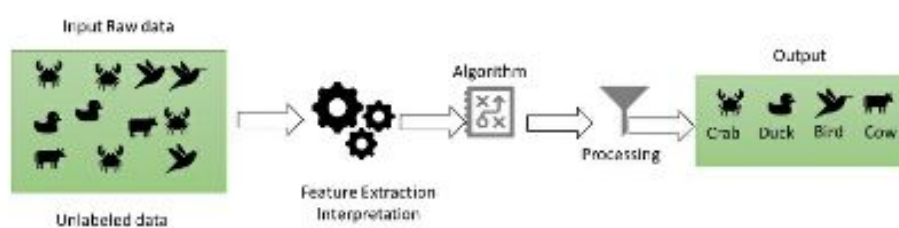


Figure 1.4.2: Concept of Unsupervised learning

Example of unsupervised learning:

Let's say you have a dataset of customer purchase history, but the data is unlabeled, which means you don't know which products or categories each purchase belongs to. With unsupervised learning, you could use clustering algorithms to group similar purchases together based on patterns or similarities in the data. This would allow you to identify segments of customers who tend to purchase similar products, which could be used to target marketing campaigns or make recommendations to customers.

Another example of unsupervised learning is anomaly detection, which is used to identify rare or unusual patterns in the data. Anomaly detection algorithms can be trained on unlabeled data and learn to recognize patterns that are significantly different from the norm. For example, an anomaly detection algorithm could be used to detect fraudulent credit card transactions by identifying patterns that are significantly different from typical transactions.

1.4 Semi-supervised learning

Semi-supervised learning is a type of machine learning that combines elements of both supervised and unsupervised learning. In semi-supervised learning, the algorithm is trained on a mixture of labeled and unlabeled data. The goal of semi-supervised learning is to leverage the available labeled data to improve the quality of the unsupervised learning process.

Let's say you have a dataset of customer reviews for a product, but only a small percentage of the reviews have been labeled as positive or negative. With semi-supervised learning, you could use the labeled data to train a classifier to recognize positive and negative reviews, and then use the classifier to label the remaining unlabeled data. This would allow you to significantly increase the amount of labeled data, which could improve the performance of the classifier.



Figure 1.4.3: Flow chart of Semi supervised learning



In machine learning, a classifier is an algorithm that learns to assign categories or labels to input data based on patterns and relationships within the data.

Another example of semi-supervised learning is in image recognition. Let's say you have a dataset of images, but only a small percentage of the images have been labeled with the correct object or class. With semi-supervised learning, you could use the labeled data to train a classifier to recognize the

object or class, and then use the classifier to label the remaining unlabeled images. This would allow you to significantly increase the amount of labeled data, which could improve the performance of the classifier.

1.5 Reinforcement learning

Reinforcement learning is a type of machine learning in which an agent learns to make decisions in an environment through trial and error. The goal of the agent is to maximize a cumulative reward signal, which is given by the environment in response to the agent's actions.



In reinforcement learning, an agent is the entity that interacts with an environment to learn how to make decisions and take actions that maximize a cumulative reward signal. The agent receives information from the environment in the form of observations and rewards, and then selects actions based on that information.

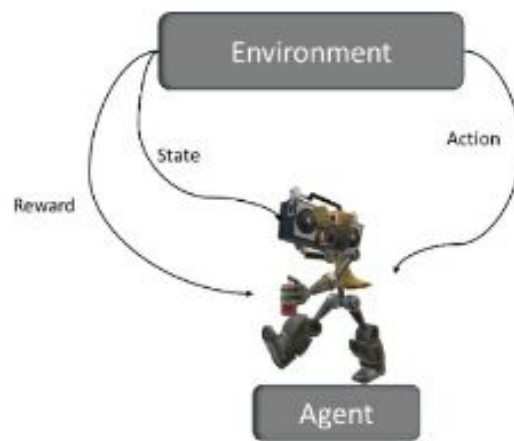


Figure 1.5.1: Agent and reward in Reinforcement learning

Imagine you are teaching a robot to navigate through a maze. At the start of the training, the robot has no prior knowledge of the maze and doesn't know which actions will lead it to the goal. The robot's goal is to reach the goal as quickly as possible while avoiding obstacles and dead-ends.

To train the robot using reinforcement learning, you would first define a reward function that assigns a positive reward when the robot reaches the goal and a negative reward when the robot collides with an obstacle or reaches a dead-end. The robot would then explore the maze by taking random actions and observing the rewards it receives for each action.

Over time, the robot will learn to associate certain actions with positive rewards and others with negative rewards, allowing it to develop a policy for navigating through the maze. As the robot gains more experience, its policy will become more refined, and it will be able to navigate through the maze more efficiently.

CHAPTER 2

SUPERVISED LEARNING

2.1 Introduction

Supervised learning is a branch of machine learning where algorithms learn patterns and relationships from labeled training data to make predictions or decisions. In the context of supervised learning, two primary tasks are classification and regression.

2.1.1. Classification Algorithms

In classification, the goal is to categorize input data into predefined classes or labels. Several powerful algorithms are employed for classification tasks, including Logistic Regression, K-Nearest

Neighbors (KNN), Support Vector Machines (SVM), Decision Trees, Random Forest, Naive Bayes, and various Gradient Boosting Algorithms such as XGBoost and LightGBM.

2.1.2. Regression Algorithms

Regression, on the other hand, involves predicting a continuous outcome or numerical value. Linear Regression, Ridge Regression, and Lasso Regression are commonly used for linear relationships, while Decision Trees, Random Forest, and Gradient Boosting Algorithms like XGBoost and LightGBM can be adapted for regression tasks.

These algorithms serve as the foundation for building predictive models in diverse fields such as finance, healthcare, and industry, where understanding and leveraging patterns within data are crucial for making informed decisions.



*Figure 2.1: Types of Classification Algorithms
in Supervised Learning*



*Figure 2.2: Types of Regression Algorithms
in Supervised Learning*

Lets looks the Supervised Learning algorithms in general

- **Linear Regression:** A simple algorithm that models the relationship between input features and output values as a linear function. It is often used for predicting continuous numerical values.
- **Logistic Regression:** A classification algorithm that models the probability of a binary outcome as a logistic function of the input features. It is often used for binary classification problems.

-
- **Decision Trees:** A tree-based algorithm that recursively partitions the input space into subsets based on the input features. It is often used for classification and regression problems.
 - **Random Forest:** A type of ensemble learning algorithm that combines multiple decision trees to improve prediction accuracy. It is often used for classification and regression problems.
 - **Naive Bayes:** A probabilistic algorithm that models the conditional probabilities of each input feature given the class label. It is often used for classification problems.
 - **Support Vector Machines (SVM):** A powerful algorithm that finds a hyperplane that separates the input space into different classes. It is often used for classification and regression problems.

-
- **Neural Networks:** A family of algorithms that use artificial neural networks to model complex relationships between input features and output values. They are often used for classification, regression, and other tasks, such as image and speech recognition.
 - **K-Nearest Neighbors (KNN):** It stands as a cornerstone in the landscape of machine learning, embodying fundamental principles that have garnered widespread adoption across diverse domains. As a non-parametric and instance-based algorithm, KNN operates on the intuitive idea that objects in similar contexts are likely to share similar characteristics.

Linear Regression

Linear regression in machine learning is a technique for predicting a continuous output variable (also known as the dependent variable) based on

one or more input variables (also known as the independent variables or features).

The goal of linear regression in machine learning is to find a linear relationship between the input variables and the output variable that can be used to make predictions on new data. This is done by learning the coefficients (also known as weights) that define the linear equation that best fits the data.

The equation for a simple linear regression with one independent variable is:

$$y = mx + b$$

Where:

- *y is the output variable*
- *x is the input variable*
- *m is the coefficient for the input variable, which represents the change in y for a one-unit increase in x*

-
- b is the intercept, which represents the value of y when x is zero

In machine learning, we typically have multiple input variables, so the equation becomes:

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

Where:

- y is the output variable
- $x_1, x_2, \dots, x_n$ are the input variables (features)
- b_0 is the intercept (also known as the bias term)
- $b_1, b_2, \dots, b_n$ are the coefficients for the input variables (features), which represent the change in y for a one-unit increase in each input variable.

To find the values of the coefficients that best fit the data, we use an optimization algorithm that

minimizes the difference between the predicted values and the actual values of the output variable, which is also known as the cost function or loss function.

Once the coefficients have been learned, the model can be used to make predictions on new data by simply plugging in the values of the input variables into the equation and solving for the output variable.

2.7.3 Example to build a linear regression model

Let's say you work for a real estate company, and you want to predict the sale price of a house based on its size (in square feet) and the number of bedrooms it has. You have a dataset of houses that have sold in a given area, and for each house, you have the following data:

- Size: the size of the house in square feet
- Number of bedrooms: the number of bedrooms in the house

-
- Sale price: the sale price of the house in dollars

To build a linear regression model, you would use the size and number of bedrooms as the independent variables (x), and the sale price as the dependent variable (y). You would then use a training dataset to estimate the coefficients b_0 , b_1 , and b_2 in the equation:

$$y = b_0 + b_1x_1 + b_2x_2$$

where x_1 is the size of the house and x_2 is the number of bedrooms.

The goal is to find the values of b_0 , b_1 , and b_2 that minimize the difference between the actual sale prices and the predicted sale prices. Once the model is trained, you can use it to predict the sale price of a new house given its size and number of bedrooms.

For example, if you have a new house that is 2,000 square feet and has 3 bedrooms, you can use the equation to predict the sale price:

predicted sale price = $b_0 + b_1 * 2000 + b_2 * 3$

The result of the prediction will depend on the coefficients you obtained from the training dataset.

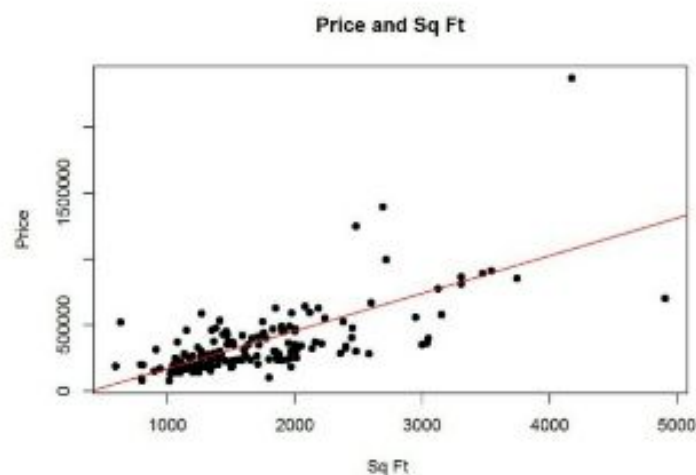


Figure 2.2.1.1: Output representation of the Linear regression example



The equation $y = mx + b$ is the equation for a simple linear regression model, where m is the slope of the line (i.e. the change in y for a one-unit increase in x), and b is the y-intercept (i.e. the value of y when x is zero).

*The equation $y = b_0 + b_1 * x$ is the equation for a multiple linear regression model, where b_0 is the y-intercept, and b_1 is the coefficient for the independent variable x . In a multiple linear regression model,*

there can be multiple independent variables, and each one would have its own coefficient.

2.7.4 A Python program with above equation to predict the sales price of a given House

```
# Import the necessary libraries
```

```
import pandas as pd
```

```
from sklearn.linear_model import Linear-  
Regression
```

```
# Load the dataset
```

```
df = pd.read_csv('houses.csv')
```

```
# Split the data into input (X) and output (y)  
variables
```

```
X = df[['size', 'bedrooms']]
```

```
y = df['price']
```

```
# Create a linear regression object
```

```
model = LinearRegression()
```

```
# Fit the model to the data
```

```
model.fit(X, y)
```

```
# Predict the sale price of a new house with  
2,000 square feet and 3 bedrooms
```

```
new_house = [[2000, 3]]
```

```
predicted_price = model.predict(new_house)
```

```
# Print the predicted sale price
```

```
print('The predicted sale price for a house  
with 2,000 square feet and 3 bedrooms is $  
{:,.2f}'.format(predicted_price[0]))
```

In this example, we first load the dataset of house sales from a CSV file called 'houses.csv'. We then split the data into input variables (X), which consist of the size and number of bedrooms for each house, and the output variable (y), which is the sale price.

We create a linear regression object and fit the model to the data using the **fit()** method. We then use the **predict()** method to predict the sale price

of a new house with 2,000 square feet and 3 bedrooms, and print the predicted price to the console.



***LinearRegression()** is a class in the scikit-learn library in Python that provides an implementation of the linear regression algorithm. It is used to fit a linear model to a given dataset and make predictions based on that model.*

*To use **LinearRegression()**, you first need to create an instance of the class by calling **LinearRegression()** with any desired parameters. Once the object is created, you can fit the linear model to a dataset using the **fit()** method, which takes two arguments: the input data (X) and the output data (y).*

2.7.5 Linear regression Applications in real-world

1. Predictive modeling: Linear regression can be used for predictive modeling tasks, such as predicting housing prices or stock prices based on historical data.

-
2. Customer segmentation: Linear regression can be used to segment customers based on various attributes such as age, income, and buying patterns. This can help businesses to target their marketing campaigns more effectively.
 3. Risk assessment: Linear regression can be used to assess the risk associated with various activities, such as lending money or underwriting insurance policies.
 4. Demand forecasting: Linear regression can be used to forecast demand for products and services based on historical data and other relevant factors such as seasonality and economic indicators.
 5. Quality control: Linear regression can be used to monitor quality control in manufacturing processes. It can help identify factors that are affecting product quality and take corrective action.

-
6. Performance analysis: Linear regression can be used to analyze the performance of various systems or processes, such as supply chain management or customer service.
 7. Resource allocation: Linear regression can be used to optimize the allocation of resources, such as staff, equipment, and inventory, to maximize efficiency and profitability.

2.8 Logistic regression

Logistic regression is a machine learning algorithm used for classification problems, where the goal is to predict a binary outcome, such as "yes" or "no", "true" or "false", "spam" or "not spam", etc. It works by fitting a logistic function to the data, which maps input features to the probability of the output variable being one or the other.

Imagine you want to predict whether an email is spam or not based on the presence of cer-

tain words in the email. You start by collecting a dataset of emails, where each email is labeled (supervised Learning) as either "spam" or "not spam", and the presence or absence of certain words is recorded as input features.

Next, you use logistic regression to build a model that learns to predict the probability of an email being spam based on the input features. The logistic regression model works by taking the input features, computing a linear combination of them, and then passing that linear combination through a logistic function to produce a probability.

The logistic function is a type of S-shaped curve that maps any input value to a value between 0 and 1. The logistic function has the property that it approaches 0 as the input value approaches negative infinity, and approaches 1 as the input value approaches positive infinity. This is useful for classification problems, where we want to predict a binary outcome that can only take on values of 0 or 1.

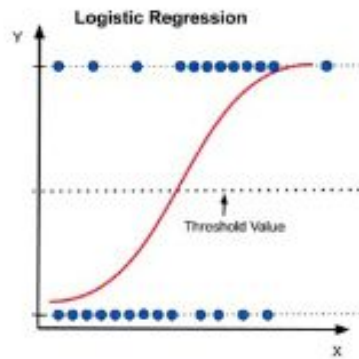


Figure 2.3.1: Graphical representation of Logistic regression

After training the logistic regression model on the email dataset, you can use it to predict the probability of an email being spam based on its input features. For example, if the model predicts a probability of 0.9 for an email, you can classify it as spam with a high degree of confidence.

In logistic regression, the linear function of the independent variables is typically modeled using the following equation:

$$\text{logit}(p) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$



where p is the probability of the dependent variable taking the value 1, β_0 is the intercept, β_1 to β_n are the coefficients or weights of the independent variables x_1 to x_n , and the logit function is defined

as the natural logarithm of the odds of the dependent variable taking the value 1, given the values of the independent variables.

Logistic regression is a commonly used algorithm in machine learning for binary classification tasks, such as spam detection, medical diagnosis, and credit risk assessment. It is simple to implement, interpretable, and can handle both numerical and categorical independent variables.

2.8.1 Python program that uses logistic regression to predict whether an email is spam or not based on its content

```
import pandas as pd

from sklearn.feature_extraction.text import
CountVectorizer

from sklearn.linear_model import Logisti-
cRegression

# Load the email dataset

data = pd.read_csv('spam.csv')
```

Split the data into input features and target variable

X = data['text']

y = data['spam']

Convert the text data into a matrix of word counts

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(X)

Train a logistic regression model on the data

model = LogisticRegression()

model.fit(X, y)

Test the model on new data

new_emails = ['Get rich quick!', 'Hi John, are you free for lunch today?']

new_emails = vectorizer.transform(new_emails)

```
predictions = model.predict(new_emails)

print(predictions)
```

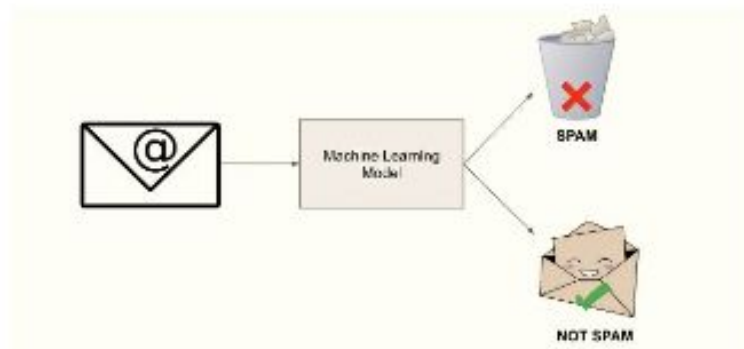


Figure 2.3.1: Representation of an Email classification

In this example, we first load the email dataset and split it into input features (the email text) and target variable (whether the email is spam or not). We then use the **CountVectorizer** class from Scikit-learn to convert the text data into a matrix of word counts. This converts each email into a vector of numerical features that the logistic regression model can use.



*We then train a logistic regression model on the data using the **LogisticRegression** class from*

*Scikit-learn. Once the model is trained, we can use it to predict the classification of new emails by first converting the new email text into a matrix of word counts using the same **CountVectorizer** instance, and then passing it to the **predict** method of the logistic regression model.*

In this example, we test the model on two new emails, one of which is spam and one of which is not. The output of the program will be an array of binary predictions, where 1 indicates that the email is spam and 0 indicates that it is not.

2.9 Decision trees

Decision trees are a type of algorithm used in machine learning and data mining to build models that can make predictions or classifications. In simple terms, a decision tree is like a flow-chart that makes decisions based on the features or characteristics of the data it is analyzing. It is called a tree because it is structured like a tree, with branches and nodes that split into smaller branches and nodes.

To illustrate this, let's take a simple example of a decision tree used for classifying whether a fruit is an apple or an orange based on its color, diameter, and weight.

Here is how the decision tree would work:

1. Start at the root node.
2. Ask if the fruit is red or orange.
3. If the fruit is red, ask if its diameter is greater than 3 inches.
4. If the diameter is greater than 3 inches, classify it as an apple.
5. If the diameter is less than or equal to 3 inches, ask if its weight is greater than 150 grams.
6. If the weight is greater than 150 grams, classify it as an orange.
7. If the weight is less than or equal to 150 grams, classify it as an apple.

8. If the fruit is orange, classify it as an orange.

The decision tree is constructed by analyzing a dataset of labeled examples of fruits and using the features of each example to create branches and nodes. The tree is then used to make predictions on new, unlabeled examples of fruits.

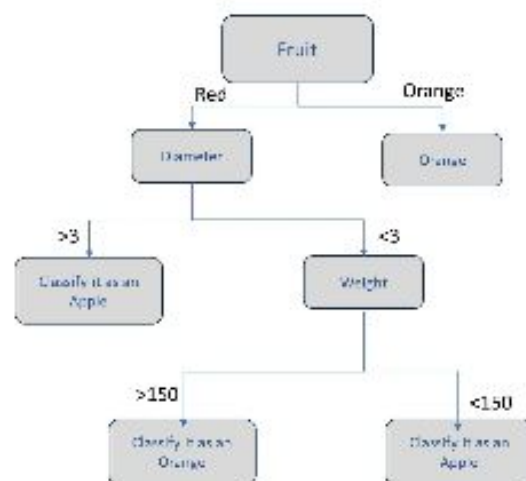


Figure 2.4.1: Decision tree to classify Orange and Apple

Decision trees are useful because they are easy to understand and interpret, and they can handle both categorical and numerical data. The process of training a decision tree involves recursively partitioning the training data based on the input features until the resulting subsets of data at each

leaf node are as pure as possible in terms of class or regression value. This process is known as recursive binary splitting and involves selecting the best feature and split point at each node based on a certain criterion.



Recursively partitioning the data refers to the process of dividing the training data into smaller subsets, based on the values of the input features, in order to create a decision tree. At each step of the partitioning process, the algorithm selects the best feature and split point that will divide the data into two or more subsets that are as pure as possible in terms of class or regression value.

The recursive aspect of this process means that each subset that is created at a given step can be further partitioned in the same way, using a new feature and split point. This process continues until a stopping criterion is reached, which is typically when the data at a given leaf node is as pure as possible in terms of class or regression value, or when a pre-defined maximum tree depth is reached.

The goal of this process is to create a decision tree that can accurately predict the target variable (i.e. the class or regression value) for new, unseen instances. This is done by constructing a tree that is both accurate and interpretable. The accuracy of the tree depends on the choice of splitting criterion and the complexity of the tree, while interpretability is achieved by ensuring that the resulting tree is easy to understand and visualize.

2.9.1 Example of how to create a decision tree using scikit-learn, a machine learning library in Python

```
from sklearn.datasets import load_iris

from sklearn.tree import DecisionTreeClassifier

from sklearn.model_selection import
train_test_split

# Load the iris dataset
```

```
iris = load_iris()

# Split the data into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size=0.3, random_state=42)

# Create a decision tree classifier with a maximum depth of 3

tree = DecisionTreeClassifier(max_depth=3)

# Train the decision tree classifier on the training data

tree.fit(X_train, y_train)

# Predict the classes of the test data

y_pred = tree.predict(X_test)

# Calculate the accuracy of the decision tree classifier

accuracy = tree.score(X_test, y_test)
```

```
# Print the accuracy of the decision tree classifier  
  
print("Accuracy:", accuracy)
```

In this example, we first load the iris dataset using the **load_iris** function from scikit-learn. We then split the data into training and testing sets using the **train_test_split** function. Next, we create a decision tree classifier with a maximum depth of 3 using the **DecisionTreeClassifier** class.

We train the decision tree on the training data using the **fit** method, and then use the trained tree to predict the classes of the test data using the **predict** method. Finally, we calculate the accuracy of the decision tree using the **score** from a single class. This is done to keep the tree simple and interpretable.

2.9.2 Applications of Decision Tree model

The Decision Tree model is a popular and widely-used machine learning algorithm that is used for

both classification and regression tasks. Here are some common applications of the Decision Tree model:

- 1. Customer segmentation: Decision trees can be used to segment customers based on their behavior, preferences, or demographic characteristics. This information can be used to create targeted marketing campaigns or to personalize product recommendations.*
- 2. Fraud detection: Decision trees can be used to identify fraudulent transactions or activities by identifying patterns in the data that are indicative of fraud.*
- 3. Medical diagnosis: Decision trees can be used to diagnose medical conditions by analyzing patient data such as symptoms, test results, and medical history.*
- 4. Credit risk analysis: Decision trees can be used to evaluate credit risk by analyzing factors*

such as income, credit score, and employment history.

- 5. Predictive maintenance: Decision trees can be used to predict when maintenance is required for a machine or system based on historical data such as usage patterns, maintenance records, and sensor data.*
- 6. Image classification: Decision trees can be used to classify images based on features such as color, texture, and shape.*
- 7. Recommendation systems: Decision trees can be used to build recommendation systems that suggest products, services, or content to users based on their preferences, behavior, or past interactions.*
- 8. Environmental monitoring: Decision trees can be used to analyze data from environmental sensors to monitor factors such as air quality, water quality, and temperature.*

-
9. *Stock price prediction: Decision trees can be used to predict stock prices by analyzing factors such as company financials, economic indicators, and market trends.*
 10. *Sentiment analysis: Decision trees can be used to analyze text data such as customer reviews, social media posts, or news articles to determine the sentiment (positive, negative, or neutral) of the content.*

Random Forest Algorithm

Random Forest is a popular ensemble learning algorithm that is widely used in machine learning for both classification and regression tasks. The name "Random Forest" comes from the fact that the algorithm is a combination of two concepts: decision trees and randomness.



Ensemble learning is a machine learning technique that involves combining multiple models to improve the overall performance of a machine learning algorithm. Ensemble learning algorithms are

widely used in many applications, including classification, regression, and clustering.

The "forest" in Random Forest refers to the fact that the algorithm is a collection of decision trees, each of which is trained on a random subset of the training data. The idea is that by combining multiple decision trees, each trained on a slightly different subset of the data, the resulting model will be more robust and less prone to overfitting.

The "random" in Random Forest refers to the fact that each decision tree is trained on a random subset of the features in the data. This helps to reduce the correlation between the trees and can lead to better performance. In addition, the use of random sampling and bootstrapping techniques in the training process also introduces an element of randomness into the algorithm.

Random Forest algorithm works by combining multiple Decision Trees to make a more accurate prediction. The basic steps in the Random Forest algorithm are as follows:

-
1. **Data Preparation:** The first step is to prepare the data for the Random Forest algorithm. This involves splitting the data into training and testing sets, and performing any necessary pre-processing, such as scaling or normalization.
 2. **Building Trees:** Next, the Random Forest algorithm builds a large number of Decision Trees on the training data. Each Decision Tree is built using a random subset of the features in the data, which helps to reduce overfitting. In addition, the Random Forest algorithm uses a technique called bagging (bootstrap aggregating), where each Decision Tree is built on a different random sample of the training data with replacement.
 3. **Voting:** Once the Decision Trees are built, the Random Forest algorithm uses them to make predictions on the testing data. Each Decision Tree produces a prediction, and

the final prediction is determined by a majority vote among all the trees.

Example of Random Forest Algorithm

Imagine you are trying to predict whether a person is likely to buy a new car. To do this, you have a dataset of information about a large number of customers, such as their age, income, and past buying behavior. You want to build a machine learning model that can use this data to accurately predict which customers are most likely to buy a car.



One way to build such a model is to use a decision tree. A decision tree is a flowchart-like structure in which each internal node represents a test on a feature, each branch represents the outcome of the test, and each leaf node represents a prediction. However, a single decision tree may not be accurate enough to make accurate predictions for every customer in the dataset. To improve accuracy, we can use the Random Forest algorithm.

Random Forest is an ensemble learning algorithm that combines many decision trees to create a more accurate and robust model. The idea is to create many different decision trees, each trained on a random subset of the data and a random subset of the features.

During training, the algorithm selects a random sample of the data (with replacement) and a random subset of the features to use for each decision tree. This process is repeated many times to create a forest of decision trees.

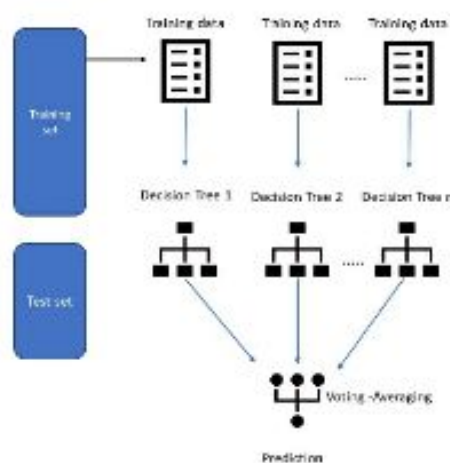


Figure 2.5.1.1: Illustration of Random Forest algorithm

When we want to make a prediction for a new customer, we pass their data through each of the decision trees in the forest. Each tree makes a pre-

diction based on its own set of features and data points. The final prediction is then determined by aggregating the predictions of all the decision trees. This helps to reduce the risk of overfitting and improve the accuracy of the model.

2.10.2 Example of building a Random Forest model in Python

```
# Import the required libraries
```

```
import pandas as pd
```

```
from sklearn.ensemble import Random-  
ForestClassifier
```

```
from sklearn.model_selection import  
train_test_split
```

```
from sklearn.metrics import accuracy_score
```

```
# Load the dataset
```

```
df = pd.read_csv('car_purchase.csv')
```

```
# Split the dataset into features and target
```

```
X = df.iloc[:, :-1]

y = df.iloc[:, -1]

# Split the dataset into training and testing
sets

X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2, ran-
dom_state=42)

# Create a Random Forest classifier object

rf = RandomForestClassifier(n_estima-
tors=100, random_state=42)

# Train the model on the training data

rf.fit(X_train, y_train)

# Evaluate the model on the testing data

y_pred = rf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print('Accuracy:', accuracy)
```

In this example, we first import the required libraries, including pandas for data manipulation, scikit-learn's Random Forest classifier, and various utility functions. We then load the car purchase dataset, which contains information about customers' age, gender, income, and whether they purchased a car.

Next, we split the dataset into features and target, where the features are all columns except the last (which contains the target variable). We then split the dataset into training and testing sets using the **train_test_split** function.

We then create a Random Forest classifier object with 100 trees using the **RandomForestClassifier** function, and train the model on the training data using the **fit** method of the classifier object.

*The **n_estimators** parameter sets the number of decision trees in the Random Forest, while the **random_state** parameter sets the seed for the random number generator used by the algorithm.*



*The **n_estimators** parameter is an important hyperparameter in Random Forest that controls the number of trees in the ensemble. Increasing the number of trees can lead to better performance, but at the cost of increased computation time and the risk of overfitting. In this example, 100 trees were chosen as a relatively large number that would likely yield good performance without excessive computational cost.*

*The **random state** parameter is used to ensure that the results of the Random Forest are reproducible. By setting a fixed seed for the random number generator, we can ensure that the same sequence of random numbers is generated each time we run the code, leading to consistent results across different runs. In this example, the value of 42 was chosen arbitrarily as the seed for the random number generator. It could have been any non-negative integer value.*

2.11 The Naive Bayes algorithm

The Naive Bayes algorithm is a probabilistic classification algorithm that uses Bayes' theorem to predict the class of a new observation based on its

features. The algorithm is called "naive" because it assumes that the features are conditionally independent given the class, even though this assumption may not always hold in practice.

Naive Bayes is a simple yet powerful algorithm for classification tasks in machine learning. The algorithm is based on Bayes' theorem and assumes that the features are independent of each other. The Naive Bayes algorithm can be used for both binary and multiclass classification problems. It works by calculating the conditional probability of each class given the features and selecting the class with the highest probability as the predicted class.

How the Naive Bayes algorithm works:

- Collect a dataset of observations, each with a set of features and a corresponding class label. For example, we might collect a dataset of emails, with features such as the sender's email address, the subject line, and the body text, and a class label of "spam" or "not spam".

-
- Calculate the prior probabilities of each class, which represent the relative frequency of each class in the dataset. For example, if we have 1000 emails in our dataset, and 300 of them are spam, then the prior probability of the spam class is $300/1000 = 0.3$, and the prior probability of the not-spam class is $1-0.3 = 0.7$.
 - For each feature and each class, calculate the conditional probability of the feature given the class. This represents the probability of observing a particular feature given that the email belongs to a particular class. For example, we might calculate the conditional probability of the word "free" given the spam class, which would be the number of spam emails that contain the word "free" divided by the total number of spam emails.
 - Given a new observation with a set of features, calculate the probability of the observation belonging to each class using Bayes'

theorem, which states that the probability of a hypothesis (in this case, the class) given the evidence (the features) is proportional to the probability of the evidence given the hypothesis times the prior probability of the hypothesis. In other words, we calculate the product of the conditional probabilities of each feature given the class, and multiply it by the prior probability of the class.

- Finally, we predict the class with the highest probability as the output of the algorithm.

Example spam emails detection

```
import os
```

```
import numpy as np
```

```
from sklearn.naive_bayes import MultinomialNB
```

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
from sklearn.model_selection import
train_test_split

from sklearn.metrics import accuracy_score

# Load the email data into memory

emails = []

labels = []

for label in ["ham", "spam"]:

    path = os.path.join("data", label)

    for filename in os.listdir(path):

        with open(os.path.join(path, filename),
"r", encoding="ISO-8859-1") as f:

            emails.append(f.read())

            labels.append(label)

# Convert the email data into a numerical rep-
resentation using a bag-of-words model

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(emails)
```

```
y = np.array(labels)

# Split the dataset into training and testing
sets

X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2, ran-
dom_state=42)

# Create a Naive Bayes classifier object

clf = MultinomialNB()

# Train the classifier on the training data

clf.fit(X_train, y_train)

# Use the trained classifier to predict the class
labels for the test data

y_pred = clf.predict(X_test)

# Evaluate the accuracy of the classifier

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:      {:.2f}%".format(accu-
racy*100))
```

In this example, we first load the email data from two directories, one for spam emails and one for non-spam (ham) emails. We then convert the email data into a numerical representation using a bag-of-words model, which counts the number of occurrences of each word in each email. This is done using the **CountVectorizer** class from scikit-learn.

Next, we split the dataset into training and testing sets using the **train_test_split** function from scikit-learn. We create a **MultinomialNB** classifier object, which is a type of Naive Bayes classifier that is suitable for count-based data such as the bag-of-words representation. We train the classifier on the training data using the **fit** method, and then use it to predict the class labels for the test data using the **predict** method.



*The **MultinomialNB** classifier in scikit-learn is implemented using the **MultinomialNB** class, which is part of the **naive_bayes** module. To use the*

***MultinomialNB** classifier, you first need to convert your data into a count-based representation, such as the bag-of-words model, using a vectorizer such as the **CountVectorizer** class from scikit-learn. Then, you can create a **MultinomialNB** object and train it on your training data using the **fit** method. Once the classifier is trained, you can use it to predict the class labels for new data using the **predict** method.*

2.12 Support Vector Machines (SVMs)

Support Vector Machines (SVMs) are a type of machine learning algorithm used for classification and regression analysis. The primary goal of SVMs is to find the best decision boundary that separates data points into different classes while maximizing the margin between the decision boundary and the closest data points of each class.



The name "support vector machine" comes from the fact that the algorithm finds the support vectors that play a crucial role in defining the hy-

perplane that separates the two classes. The term "support vector" refers to the fact that these points support the hyperplane in the sense that if we were to remove any of them, the position and orientation of the hyperplane would change. The support vectors are the data points that are closest to the decision boundary, and they are used to determine the optimal hyperplane.

Imagine you have a dataset of points, where each point has some features and belongs to one of two classes, say 'A' or 'B'. The goal of SVM is to find a line (in 2D) or hyperplane (in higher dimensions) that separates the points of class 'A' from those of class 'B' with maximum margin. The margin is the distance between the hyperplane and the closest points of each class, which are called support vectors.



A dataset of points refers to a collection of data points, where each data point has some features or attributes and belongs to a specific class or cat-

egory. For example, in a dataset of customer information for a bank, each data point may represent a customer, and the attributes or features may include their age, income, credit score, and so on. The class or category may indicate whether the customer is likely to default on a loan or not.

In the context of SVMs, the dataset of points typically contains two classes or categories, and the goal is to find a way to separate the points of one class from the other. This can be done by finding a hyperplane that maximizes the margin between the two classes while minimizing the classification error.

2.12.1 Example of how SVMs work for a binary classification problem

Suppose we have a binary classification problem where we want to classify emails as spam or not spam. We have a dataset of labeled emails, where each email is labeled as either spam or not spam. To train an SVM for this problem, we first need to

represent the emails as vectors of features. Some possible features could be the number of words in the email, the number of exclamation points in the email, or the presence of certain keywords, such as "free" or "money." Once we have represented the emails as vectors of features, we can train the SVM to find a hyperplane that separates the spam emails from the not spam emails. The hyperplane is a line or plane in the feature space that divides the data into two classes.

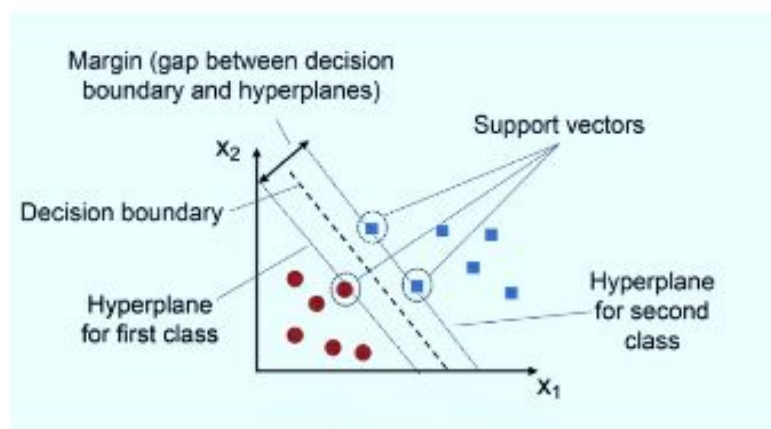


Figure 2.7.1.1 : SVMs work for a binary classification problem

The SVM tries to find the hyperplane that has the largest margin between the two classes. The margin is the distance between the hyperplane and the nearest data points from each class. The SVM also tries to minimize the number of data points that

fall on the hyperplane. These data points are called support vectors. Once the SVM is trained, we can use it to classify new emails as spam or not spam. To do this, we simply need to project the new email onto the hyperplane. If the projection falls on the spam side of the hyperplane, then we classify the email as spam. Otherwise, we classify the email as not spam.

2.7.2. Example of an SVM classifier used to classify spam emails

Suppose we have the following two spam emails:

Email 1: "Free money! Click here now!"

Email 2: "Win a new car! Reply to this email for your chance to win!"

We can represent these emails as vectors of features as follows:

Email 1: [number of words: 10, number of exclamation points: 2, presence of keyword "free": yes]

Email 2: [number of words: 15, number of exclamation points: 3, presence of keyword "win": yes]

We can then train an SVM to find a hyperplane that separates these two emails. The hyperplane might look something like this:

$$y = mx + b$$

where y represents the distance from the hyperplane to the spam side of the hyperplane, m is the slope of the hyperplane, and b is the y-intercept of the hyperplane.

Once the SVM is trained, we can use it to classify new emails as spam or not spam. For example, if we receive a new email with the following features:

[number of words: 12, number of exclamation points: 1, presence of keyword "free": no]

We can project this email onto the hyperplane. If the projection falls on the spam side of the hyperplane, then we classify the email as spam. Otherwise, we classify the email as not spam.

2.8.3 Python code example for training an SVN (Support Vector Machine) machine learning model on a simple classification problem

```
from sklearn import datasets

from    sklearn.model_selection    import
train_test_split

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score

# Load dataset

iris = datasets.load_iris()

# Split data into training and testing sets

X_train,  X_test,  y_train,  y_test  =
train_test_split(iris.data, iris.target, test_
size=0.2)

# Create an instance of SVM classifier with a
linear kernel
```

```
clf = SVC(kernel='linear')  
  
# Train the SVM classifier on the training set  
clf.fit(X_train, y_train)  
  
# Predict the classes of the test set  
y_pred = clf.predict(X_test)  
  
# Evaluate the accuracy of the classifier  
accuracy = accuracy_score(y_test, y_pred)  
  
print('Accuracy:', accuracy)
```

In this example, we use the *iris* dataset, which is a classic machine learning dataset that contains measurements of various features of iris flowers, along with their corresponding species. We split the dataset into training and testing sets, and create an instance of the SVM classifier with a linear kernel. We then train the classifier on the training set, predict the classes of the test set, and evaluate the accuracy of the classifier using the accuracy score function from scikit-learn's metrics module.

2.9 Neural networks

Neural networks are a type of machine learning model that is particularly well-suited for solving complex problems in supervised learning, which involves training a model on labeled data to make predictions on new, unseen data. In supervised learning, the goal is to learn a function that maps input data to output data based on a set of labeled examples.

A neural network consists of an input layer, one or more hidden layers, and an output layer. Each layer is composed of a number of nodes, or neurons, which receive input data and apply a set of weights to the inputs to generate an output value. The output of one layer is passed as input to the next layer, and the process is repeated until the output layer is reached.

In a neural network, the input layer is the first layer of neurons that receives the input data. The input layer has one neuron for each feature in the input data, and the values of these neurons rep-

resent the values of the corresponding input features.



In the context of neural networks, a neuron is a mathematical function that takes in one or more inputs and produces an output. A neural network is made up of many neurons that are arranged in layers, with each neuron in a layer connected to the neurons in the previous and next layers.

For example, if we have a dataset of images and we want to use a neural network to classify these images into different categories, the input layer would have one neuron for each pixel in the image. Each neuron in the input layer would take the value of the corresponding pixel in the image.

The input layer is sometimes also referred to as the visible layer, as the values of the neurons in this layer are directly observable from the input data. The neurons in the input layer do not perform any computations, but simply pass their input values forward to the next layer in the network. The val-

ues of the neurons in the input layer are typically normalized or scaled to ensure that they are on a similar scale, which can help the network to learn more effectively. The number of neurons in the input layer is determined by the number of features in the input data. The size of the input layer is fixed, and once the network is trained, the input layer does not change.

The weights in the network are learned through a process called backpropagation, which involves iteratively adjusting the weights based on the error between the predicted output and the actual output. During training, the model makes predictions on the training data, and the weights are adjusted to minimize the error between the predicted outputs and the actual outputs.

The weights are numbers that determine the importance of each input to the output. Imagine you have a neural network that predicts the price of a house based on its size and location. The network

would take in the size and location of the house as input, and output the predicted price.

In order to make accurate predictions, the neural network needs to learn the best weights for each input. For example, it might learn that the size of the house is more important than its location, and assign a higher weight to the size input.

During training, the neural network adjusts the weights to minimize the difference between the predicted price and the actual price. This is done using a process called backpropagation, which involves updating the weights based on the error between the predicted output and the actual output.



The goal of training a neural network is to find the best set of weights that give the most accurate predictions on new data. Once the weights have been learned, they are fixed and the neural network can be used to make predictions on new data.

The performance of a neural network depends on the values of the weights, as well as other factors

such as the number of neurons and the number of layers in the network. The weights can be fine-tuned by adjusting the hyperparameters of the network, such as the learning rate and the number of hidden layers. By tuning these hyperparameters, we can improve the accuracy of the network's predictions.

Once the neural network is trained, it can be used to make predictions on new, unseen data. The input data is passed through the network, and the output of the final layer is the predicted output.

The number of hidden layers in a neural network is the number of layers between the input and output layers. Each hidden layer contains a set of neurons, which are connected to the neurons in the previous and next layers by weights.



Determining the optimal number of hidden layers for a neural network is an important task, as it can affect the accuracy and performance of the network. However, there is no one-size-fits-all answer

to how many hidden layers a neural network should have, as it depends on the specific problem being solved and the size of the dataset.

One way to determine the number of hidden layers is to start with a small number, such as one or two, and gradually increase the number until the network achieves good performance on the validation set. The validation set is a portion of the dataset that is held out from the training process and used to evaluate the performance of the network on new data. A network with too few hidden layers may not be able to capture the complexity of the data, while a network with too many hidden layers may overfit the data and perform poorly on new data.

It's also worth noting that the number of neurons in each hidden layer is an important factor to consider when designing a neural network. The number of neurons in each hidden layer can affect the capacity of the network to learn complex relationships in the data, and the number of neurons

should be chosen based on the complexity of the problem and the size of the dataset.

2.9.4 An example of a simple 3-layer neural network

Suppose we have a dataset of images of handwritten digits, and we want to train a neural network to recognize which digit is in each image. Our neural network will have an input layer, a hidden layer, and an output layer.

1. **Input Layer:** The input layer is the first layer of the neural network and takes in the input data. In this example, each input image is a 28x28 pixel grayscale image, which we will flatten into a vector of 784 pixels. The input layer will have 784 neurons, one for each pixel in the input image.
2. **Hidden Layer:** The hidden layer is the middle layer of the neural network and is where the neural network learns to recognize patterns in the input data. The number of neu-

rons in the hidden layer is a hyperparameter that needs to be set during the design of the network. For this example, let's say we decide to use 128 neurons in the hidden layer. Each neuron in the hidden layer will take in the 784 input values and produce a single output value.

3. **Output Layer:** The output layer is the final layer of the neural network and produces the output prediction. In this example, we want to recognize which digit is in the input image, so we will have 10 neurons in the output layer, one for each possible digit (0-9). Each neuron in the output layer will produce a value between 0 and 1, representing the probability that the input image belongs to that digit.

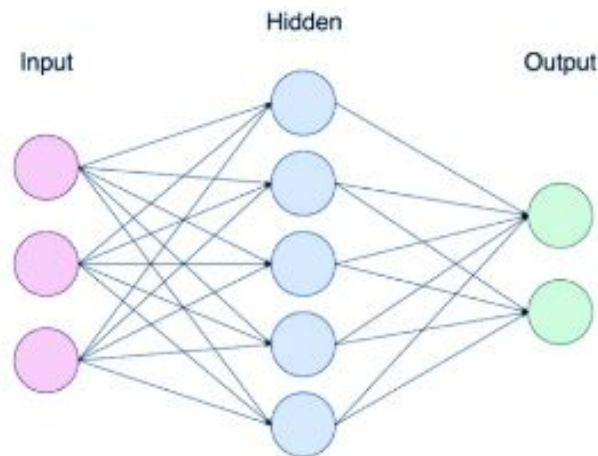


Figure 2.8.1.1: 3-layer Neural network

2.9.5 An example of how you could create a neural network classification model using Python and the Keras library

```
import numpy as np

from keras.models import Sequential

from keras.layers import Dense

# Define your training data

X_train = np.array([[0,0], [0,1], [1,0], [1,1]])

y_train = np.array([0, 1, 1, 0])

# Define your neural network model

model = Sequential()
```

```
model.add(Dense(8, input_dim=2, activation='relu'))
```

```
model.add(Dense(1, activation='sigmoid'))
```

```
# Compile the model with the appropriate  
loss function and optimizer
```

```
model.compile(loss='binary_crossentropy',  
optimizer='adam', metrics=['accuracy'])
```

```
# Train the model on the training data
```

```
model.fit(X_train, y_train, epochs=1000,  
batch_size=4)
```

```
# Test the model on some new data
```

```
X_test = np.array([[0,0], [0,1], [1,0], [1,1]])
```

```
y_test = np.array([0, 1, 1, 0])
```

```
scores = model.evaluate(X_test, y_test)
```

```
# Print out the accuracy of the model on the  
test data
```

```
print("Accuracy: %.2f%%" % (scores[1]*100))
```

-
- **Importing Libraries** - We first import the necessary libraries. **numpy** is a library for numerical computations, while **keras** is a high-level neural network API that is used to build and train deep learning models.
 - **Defining the Training Data** - We define the training data as a numpy array. In this example, we have four training samples, each with two input features (0 and 1) and one output label (0 or 1).
 - **Defining the Neural Network Model** - We define a neural network model using Keras. In this example, we're using a sequential model with one hidden layer containing 8 neurons and an output layer with one neuron. The **Dense** function is used to create the layers of the model. The **input_dim** parameter is used to specify the number of input features, and the **activation** parameter is used to specify the activation function to be used for each layer.

-
- **Compiling the Model** - We compile the model using the **compile** method. We specify the **loss** function to be used (in this case, binary cross-entropy, which is commonly used for binary classification problems) and the **optimizer** to be used (in this case, the **adam** optimizer, which is a popular optimizer for deep learning models). We also specify **metrics=['accuracy']**, which tells *Keras* to track the accuracy of the model during training.
 - **Training the Model** - We train the model on the training data using the **fit** method. We specify the number of **epochs** to train for (in this case, 1000) and the **batch_size** to be used during training (in this case, 4). During training, the model adjusts its weights to minimize the loss function and improve its accuracy.
 - **Testing the Model** - After training, we test the model on some new data (in this case,

the same data we used for training). We evaluate the model on the test data using the **evaluate** method and store the results in the **scores** variable.

- **Printing the Accuracy** - Finally, we print out the accuracy of the model on the test data using the **scores** variable. We multiply the accuracy by 100 to get a percentage value.

2.9. K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) stands as a cornerstone in the landscape of machine learning, embodying fundamental principles that have garnered widespread adoption across diverse domains. As a non-parametric and instance-based algorithm, KNN operates on the intuitive idea that objects in similar contexts are likely to share similar characteristics.

At its core, KNN relies on the notion that instances in a dataset exist within a feature space, and

their relationships can be discerned by measuring the proximity between them. This proximity, often computed using distance metrics like Euclidean distance or Manhattan distance, forms the basis for decision-making in KNN. The algorithm, in both its classification and regression variants, leverages the collective intelligence of neighboring instances to predict outcomes for new, unseen data points.



The Manhattan distance is a sum of the absolute differences between corresponding coordinates of two points. It represents the shortest path a taxi or person would take to travel from one point to another on a grid-like road network. Movement can only occur parallel to the grid lines, not diagonally.

2.9.1 KNN Working

- **Instance-Based Learning:** KNN is an instance-based or memory-based learning

algorithm. Instead of explicitly learning a model, it memorizes the training dataset.

- **Prediction by Proximity:** To predict the class or value of a new data point, KNN examines the 'k' nearest neighbors in the feature space. The majority class (for classification) or average value (for regression) of these neighbors determines the prediction.

2.9.2 Algorithmic Steps

- **Step 1: Input Data:** Start with a labeled training dataset.
- **Step 2: Define Distance Metric:** Choose a distance metric (e.g., Euclidean distance, Manhattan distance) to measure the proximity between data points.
- **Step 3: Calculate Distances:** Compute the distance between the new data point and all points in the training set.

-
- **Step 4: Identify Neighbors:** Select the 'k' nearest neighbors with the smallest distances.
 - **Step 5: Make Prediction:** For classification, assign the majority class among neighbors. For regression, predict the average of the target values.

Choosing 'k'

- **Odd vs. Even 'k':** Odd values of 'k' are often preferred to avoid ties in classification.
- **Impact of 'k':** Smaller 'k' values lead to more flexible, sensitive models, while larger 'k' values provide smoother decision boundaries.

Distance Metrics

- **Euclidean Distance:** The straight-line distance between two points in Euclidean space.

-
- **Manhattan Distance:** The sum of absolute differences between corresponding coordinates.
 - **Minkowski Distance:** Generalized form of distance that includes Euclidean and Manhattan as special cases.

2.9.3 Example of classifying Iris flowers using the K-Nearest Neighbors (KNN) algorithm

Make sure you have the necessary libraries installed before running the program. You can install them using

```
from sklearn.datasets import load_iris
```

```
from    sklearn.model_selection    import  
train_test_split
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.metrics import accuracy_score
```

```
import matplotlib.pyplot as plt

from mlxtend.plotting import plot_decision_regions

# Step 1: Load the Data

iris = load_iris()

X = iris.data # Features

y = iris.target # Target variable (species)

# Step 2: Split the Data

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 3: Choose 'k' and Distance Metric

k = 3

knn_classifier = KNeighborsClassifier(n_neighbors=k, metric='euclidean')
```

Step 4: Train the Model

```
knn_classifier.fit(X_train, y_train)
```

Step 5: Make Predictions

```
y_pred = knn_classifier.predict(X_test)
```

Step 6: Evaluate the Model

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print(f"Accuracy: {accuracy:.2f}")
```

Step 7: Visualization (Optional)

```
plt.figure(figsize=(8, 6))
```

```
plot_decision_regions(X_train, y_train, knn_classifier, legend=2)
```

```
plt.xlabel('Sepal Length (cm)')
```

```
plt.ylabel('Sepal Width (cm)')
```

```
plt.title('KNN Decision Boundaries')
```

```
plt.show()
```

CHAPTER 3

UNSUPERVISED LEARNING

3.1 Introduction

Unsupervised learning is a type of machine learning where the model learns patterns and structures in the data without explicit supervision or labeled examples. In contrast to supervised learning, where the model learns to predict output values given input features, unsupervised learning is concerned with discovering underlying patterns and structure in the data.

Suppose you have a dataset of customer purchases at a grocery store. Each row of the dataset represents a customer, and the columns represent the items they purchased. The dataset does not in-

clude any information about whether a customer is loyal or not, or any other labels that would allow us to train a supervised learning model.

So we could use unsupervised learning techniques to try to identify patterns and structure in the data. One approach might be to use clustering to group similar customers together based on their purchasing patterns.

Let's say you work for a marketing company that wants to understand its customer base better. The company has a large dataset of customer purchases, but the data is not labeled with information about which customers purchased which products. In this case, you could use unsupervised learning techniques to identify patterns and structures in the data. One approach might be group customers based on their purchase history.

There are various types of unsupervised learning techniques, some of which are:

-
1. **Clustering:** Clustering is the process of dividing the data into groups or clusters based on the similarity of the features. The goal is to group data points that are similar and to separate data points that are dissimilar.
 2. **Dimensionality Reduction:** Dimensionality reduction is the process of reducing the number of features in the data while retaining as much of the original information as possible. This is typically done by identifying the most important features and discarding the less important ones.
 3. **Anomaly Detection:** Anomaly detection is the process of identifying data points that are significantly different from the rest of the data. This can be useful for identifying outliers or detecting anomalies in data.

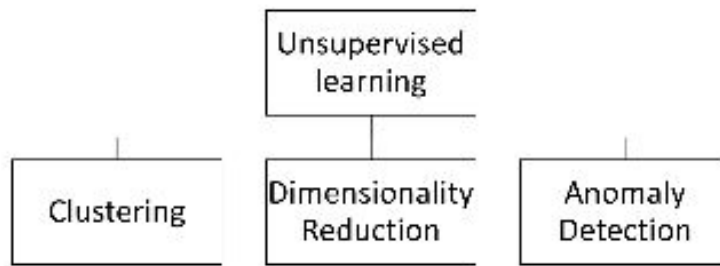


Figure 3.1 : Classification of Unsupervised learning

3.2 Clustering

Clustering is a type of unsupervised learning that involves grouping similar data points together. There are several types of clustering algorithms that can be used in unsupervised learning, each with its own strengths and weaknesses. Here are some common types of clustering algorithms:



Figure 3.2.1: Classification of Clustering algorithms

1. **K-means clustering:** This is one of the most popular clustering algorithms. It involves partitioning the data into k clusters, where k is a pre-specified number. The algorithm

iteratively assigns each data point to the nearest cluster center and updates the cluster centers based on the new assignments. The algorithm converges when the cluster assignments no longer change.

2. Hierarchical clustering: This algorithm creates a hierarchy of clusters by either starting with each data point as its own cluster and merging clusters together based on similarity, or by starting with all the data points in a single cluster and recursively splitting clusters based on dissimilarity. The result is a tree-like structure called a dendrogram, which can be cut at a certain level to obtain the desired number of clusters.
3. DBSCAN (Density-Based Spatial Clustering of Applications with Noise): This algorithm is based on the idea that clusters are areas of high density separated by areas of low density. DBSCAN clusters together points

that are close together and separates points that are far apart. The algorithm requires two parameters: the minimum number of points required to form a dense region (minPts) and the maximum distance between points in the same dense region (eps).

4. Gaussian Mixture Models: This algorithm models the data as a mixture of Gaussian distributions. The goal is to find the parameters of the Gaussians that best fit the data. The resulting clusters are defined by the Gaussians.
5. Spectral clustering: This algorithm involves first transforming the data into a lower-dimensional space using a technique such as PCA or t-SNE. It then uses k-means clustering or another clustering algorithm to group the data points in the transformed space.

K-means clustering

K-means clustering is a popular algorithm used for grouping similar data points together into clusters. The "k" in k-means refers to the number of clusters that the algorithm will create. Here is how the algorithm works:

1. Initialization: First, the algorithm randomly selects k initial cluster centers from the data points. These initial cluster centers will be used as the starting points for the k clusters.



The cluster centers, also known as centroids, represent the centers of each of the k clusters that are created during the algorithm. The cluster centers are important because they define the location of each cluster in the dataset. Data points that are assigned to the same centroid belong to the same cluster, meaning that they are similar to each other in some way.

2. Assignment: Next, each data point is assigned to the nearest cluster center. This

is done by calculating the Euclidean distance between each data point and each cluster center, and assigning the data point to the cluster with the nearest center.



Euclidean distance is commonly used in machine learning to measure the similarity between data points, such as when using the K-means clustering algorithm to group similar data points together.



3. **Update:** Once all the data points have been assigned to clusters, the cluster centers are updated. The new cluster center is calculated as the mean of all the data points assigned to that cluster. This is done separately for each cluster.

The mean of all the data points is a statistical measure that represents the average value of a set of data. It is calculated by adding up all the values in the

dataset and dividing the sum by the number of data points in the set.

4. Repeat: Steps 2 and 3 are repeated iteratively until the cluster assignments no longer change, or until a pre-specified maximum number of iterations is reached.
5. Convergence: The algorithm converges when the cluster assignments no longer change, and the final cluster centers represent the centers of the resulting clusters.

The goal of the K-means algorithm is to optimize the placement of the centroids such that the within-cluster sum of squares is minimized, which means that the data points within each cluster are as similar as possible to each other.

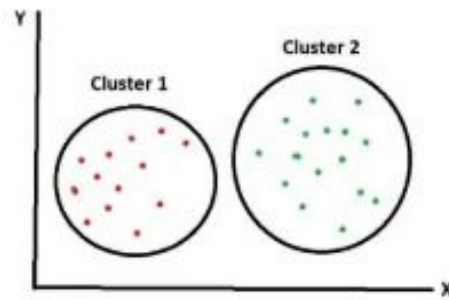


Figure 3.2.1.1: K- mean Clustering algorithm

Note that K-means clustering can be sensitive to the initial cluster centers, and the result may depend on the initial conditions. Therefore, the algorithm is often run multiple times with different initializations to ensure that the resulting clusters are robust.

Initialization

There are different ways to initialize the cluster centers in the k-means clustering algorithm. One common approach is as follows:

1. Random selection: The algorithm randomly selects k data points from the dataset to be the initial cluster centers.



"k data points" refers to the k initial cluster

centers that are randomly selected from the dataset during the initialization step. These data points serve as the starting points for the k clusters that will be created by the algorithm.

2. **K-means++ initialization:** This is a more advanced initialization technique that tries to select initial cluster centers that are farther apart from each other. The K-means++ algorithm works as follows:
 - a. Select the first cluster center randomly from the dataset.
 - b. For each subsequent cluster center, calculate the distance between each data point and the nearest cluster center that has already been selected. The probability of selecting a data point as the next cluster center is proportional to the square of this distance.
 - c. Repeat step b until k cluster centers have been selected.

This initialization technique can help to improve the performance of the k-means algorithm and reduce the risk of getting stuck in local minima.

Assignment

The "Assignment" step in the K-means clustering algorithm is the process of assigning each data point in the dataset to the nearest centroid. The goal of this step is to create k clusters of data points, with each cluster centered around a single centroid.

During the Assignment step, each data point in the dataset is evaluated to determine which centroid it is closest to based on the Euclidean distance metric. The data point is assigned to the cluster associated with the nearest centroid. In other words, if a data point is closest to the first centroid, it is assigned to the first cluster; if it is closest to the second centroid, it is assigned to the second cluster, and so on.



Euclidean distance is a measure of how far apart two points are from each other in space. It is calculated by finding the square root of the sum of the squared differences between the corresponding coordinates of the two points.

The distance metric used to determine the nearest centroid is typically Euclidean distance, as it is a commonly used distance metric and well suited for continuous data. However, other distance metrics can be used as well depending on the specific requirements of the problem at hand.

Update

The "Update" step in the K-means clustering algorithm is the process of recalculating the position of each centroid based on the mean of all the data points assigned to its cluster. The goal of this step is to improve the accuracy of the cluster centroids and better group similar data points together.

During the Update step, the position of each centroid is set to be the mean of all the data points

assigned to its cluster. This mean calculation involves summing up all the data points' values for each dimension and dividing the result by the number of data points in the cluster. This process is performed independently for each dimension.

Repeat and Convergence

The Repeat step involves executing the Assignment step to assign each data point to the nearest centroid, followed by the Update step to recalculate the position of each centroid based on the mean of all the data points assigned to its cluster. This process is repeated iteratively until a convergence criterion is met.

The convergence criterion is typically defined as a maximum number of iterations or a threshold for the change in the position of the centroids between iterations. When the change in centroid positions falls below the specified threshold or the maximum number of iterations is reached, the algorithm is said to have converged, and the Convergence step is reached. At this point, the final

cluster assignments and centroid positions are determined, and the algorithm returns these results as the output of the K-means clustering process.

3.2.4 Hierarchical clustering

Hierarchical clustering is a type of clustering algorithm that groups data points into a hierarchy of nested clusters. In hierarchical clustering, the data points are iteratively grouped together into clusters based on their similarity until all data points are in a single cluster.

There are two main types of hierarchical clustering: agglomerative and divisive. Agglomerative clustering starts with each data point in a separate cluster and iteratively merges the most similar clusters until all data points are in a single cluster. Divisive clustering, on the other hand, starts with all data points in a single cluster and iteratively splits the least similar clusters until each data point is in a separate cluster.

In agglomerative hierarchical clustering, the process starts by computing a pairwise distance matrix between all data points. Then, each data point is considered as a single cluster. At each iteration, the two closest clusters are merged together, and the distance between the new cluster and the remaining clusters is computed. This process continues until all data points are in a single cluster.

The result of the agglomerative hierarchical clustering is a dendrogram, which is a tree-like diagram that shows the order in which clusters were merged. The dendrogram can be cut at a certain level to obtain a specific number of clusters, or clusters can be identified based on the structure of the dendrogram.

In divisive hierarchical clustering, the process starts with all data points in a single cluster. At each iteration, the least similar cluster is split into two, and the distance between the new clusters and the remaining clusters is computed. This

process continues until each data point is in a separate cluster.

3.2.2.1 Example of hierarchical clustering

Let's consider an example of hierarchical clustering using the agglomerative method. Suppose we have a dataset of 8 points with 2-dimensional coordinates:

$[(2, 10), (2, 5), (8, 4), (5, 8), (7, 5), (6, 4), (1, 2), (4, 9)]$

We can start the agglomerative hierarchical clustering by computing the pairwise distances between all points. We can use the Euclidean distance as the distance metric in this example:

	1	2	3	4	5	6	7
1	0.00	5.00	6.40	3.61	5.10	4.12	8.60
2	5.00	0.00	6.08	3.16	4.12	3.16	7.81
3	6.40	6.08	0.00	4.24	1.41	2.24	9.06
4	3.61	3.16	4.24	0.00	2.24	1.41	6.08
5	5.10	4.12	1.41	2.24	0.00	1.00	7.28

6	4.12	3.16	2.24	1.41	1.00	0.00	6.08
7	8.60	7.81	9.06	6.08	7.28	6.08	0.00

The first step in agglomerative clustering is to consider each point as a separate cluster. The two closest clusters are then merged together based on the pairwise distance between them. In this example, the closest points are (2, 5) and (6, 4), with a distance of 1.0. So we merge these two points together to form a new cluster:

$[(2, 5), (6, 4)]$

Next, we update the pairwise distance matrix to include the distance between the new cluster and the remaining points:

	1	2	3	4	5	7
1	0.00	5.00	6.40	3.61	5.10	8.60
2	5.00	0.00	6.08	3.16	4.12	7.81
3	6.40	6.08	0.00	4.24	1.41	9.06

4	3.61	3.16	4.24	0.00	2.24	6.08
5	5.10	4.12	1.41	2.24	0.00	7.28
7	8.60	7.81	9.06	6.08	7.28	0.00

We repeat this process, finding the two closest clusters and merging them until all points are in a single cluster. The next closest points are (4, 9) and (5, 7), which have a distance of 1.00. We merge them to form a new cluster, which we can represent by its centroid at (4.5, 8.0).

After merging the points (4, 9) and (5, 7), the remaining clusters are:

Cluster 1: (1, 3, 6)

Cluster 2: (2)

Cluster 3: (4)

Cluster 4: (8)

Cluster 5: (4, 9, 5, 7)

The pairwise distance matrix for these clusters can be computed using the Euclidean distance be-

tween the centroids of the clusters. This matrix will be used to find the next closest clusters to merge.

3.2.5 Agglomerative method

Agglomerative hierarchical clustering is a bottom-up hierarchical clustering method in which the algorithm starts with each data point as its own cluster and then successively merges the closest pairs of clusters until all points belong to a single cluster. The result of agglomerative hierarchical

clustering is a dendrogram,  which illustrates the hierarchical relationships among clusters.

The term "agglomerative" refers to a process of clustering or forming a group by progressively combining smaller elements into larger ones based on certain criteria or similarities

The agglomerative hierarchical clustering algorithm works as follows:

1. Assign each data point to its own cluster.
2. Compute the pairwise distance matrix between all clusters.
3. Merge the two closest clusters based on the distance matrix.
4. Update the distance matrix by computing the distances between the merged cluster and the remaining clusters.
5. Repeat steps 3 and 4 until all data points are in a single cluster.

The distance between clusters is typically defined as the distance between their centroids, which can be computed using different methods, such as the mean or median of the data points in each cluster. The distance between two clusters can also be defined using different distance metrics, such as Euclidean distance or Manhattan distance.

3.3.4 Divisive hierarchical clustering

Divisive hierarchical clustering is a hierarchical clustering method that works in the opposite direction of agglomerative hierarchical clustering. In divisive hierarchical clustering, we start with all the data points assigned to a single cluster and recursively split the cluster into smaller clusters until each data point is in its own cluster.

The divisive hierarchical clustering algorithm works as follows:

1. Start with all data points in a single cluster.
2. Compute the dissimilarity matrix or pairwise distance matrix between all the points in the cluster.
3. Choose the point with the largest dissimilarity as the first cluster to split.
4. Split the selected cluster into two sub-clusters using any clustering algorithm (e.g., K-means, hierarchical clustering, etc.).

-
5. Update the dissimilarity matrix by computing the dissimilarity between the new sub-clusters and the original cluster.
 6. Repeat steps 3 to 5 until each data point is in its own cluster.

The output of divisive hierarchical clustering is a dendrogram, which shows how the original cluster was recursively split into smaller clusters. The dendrogram can be used to determine the optimal number of clusters for the data.

Divisive hierarchical clustering is less commonly used than agglomerative hierarchical clustering because it is more computationally expensive and less intuitive, as it requires choosing the point to split and deciding how to split it.

3.3.4.1 Example : Divisive hierarchical clustering

Let's consider an example of hierarchical clustering using the divisive hierarchical clustering. Suppose we have a dataset of 8 points with 2-dimen-

sional coordinates: [(2, 10), (2, 5), (8, 4), (5, 8), (7, 5), (6, 4), (1, 2), (4, 9)]

1. Start by considering all the points as a single cluster.
2. Calculate the centroid of the entire cluster, which is the mean of the coordinates of all the points:

$$\text{Centroid} = ((2+2+8+5+7+6+1+4)/8, (10+5+4+8+5+4+2+9)/8) = (4.125, 6.125)$$

3. Divide the points into two clusters, based on which side of the centroid they fall on. In this case, we can split the points vertically along the x-axis:

Cluster 1: [(2, 10), (2, 5), (1, 2), (4, 9)]

Cluster 2: [(8, 4), (5, 8), (7, 5), (6, 4)]

4. Recursively divide each cluster into sub-clusters until each cluster contains only one point.

For Cluster 1:

-
- Calculate the centroid of Cluster 1:

centroid = $((2+2+1+4)/4, (10+5+2+9)/4) = (2.25, 6.5)$

- Divide the points into two clusters based on their x-coordinates:

Sub-Cluster 1: [(2, 10), (2, 5)]

Sub-Cluster 2: [(1, 2), (4, 9)]

- Recursively divide Sub-Cluster 1 into sub-clusters:

Sub-Cluster 1a: [(2, 10)]

Sub-Cluster 1b: [(2, 5)]

Sub-Cluster 2 and Sub-Cluster 1a are already single-point clusters, so we don't need to divide them further.

- Recursively divide Sub-Cluster 2 into sub-clusters:

Sub-Cluster 2a: [(1, 2)]

Sub-Cluster 2b: [(4, 9)]

Sub-Cluster 2a and Sub-Cluster 1b are already single-point clusters, so we don't need to divide them further.

For Cluster 2:

- Calculate the centroid of Cluster 2:

centroid = $((8+5+7+6)/4, (4+8+5+4)/4) = (6.5, 5.25)$

- Divide the points into two clusters based on their x-coordinates:

Sub-Cluster 1: [(8, 4), (7, 5)]

Sub-Cluster 2: [(5, 8), (6, 4)]

- Recursively divide Sub-Cluster 1 into sub-clusters:

Sub-Cluster 1a: [(8, 4)]

Sub-Cluster 1b: [(7, 5)]

Sub-Cluster 1a and Sub-Cluster 2 are already single-point clusters, so we don't need to divide them further.

- Recursively divide Sub-Cluster 2 into sub-clusters:

Sub-Cluster 2a: [(5, 8)]

Sub-Cluster 2b: [(6, 4)]

Sub-Cluster 2a and Sub-Cluster 2b are already single-point clusters, so we don't need to divide them further.

3.3.4.2 Python Code for the example to plot the dendrogram

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from scipy.cluster.hierarchy import dendrogram, linkage

# Define the dataset
```

```
data = np.array([[5, 3], [10, 15], [15, 12], [24,
10], [30, 30], [85, 70], [71, 80], [60, 78], [70,
55], [80, 91]])

# Perform hierarchical clustering using the
ward method

Z = linkage(data, method='ward')

# Plot the dendrogram

plt.figure(figsize=(10, 5))

plt.title('Hierarchical Clustering Dendro-
gram')

plt.xlabel('Data points')

plt.ylabel('Distance')

dendrogram(Z)

plt.show()
```

Here 2-dimensional numpy array is defined as the dataset. This particular dataset contains 10 data points with 2 features each. The hierarchical clus-

tering algorithm is applied to the dataset using the ward method, which minimizes the variance of the distances between the clusters being merged. This results in a linkage matrix $\mathbf{Z}$ containing information about the pairwise distances and linkages between the clusters. Finally, a dendrogram is plotted to visualize the resulting cluster hierarchy. The dendrogram shows the distances between clusters as the height of the branches, and the clusters as the leaves of the tree.

The output dendrogram is look like this

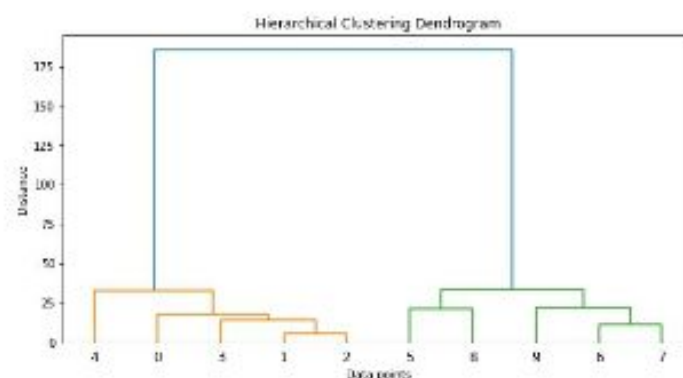
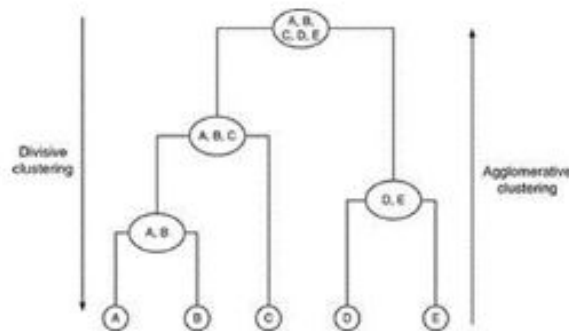


Figure 3.2.4.1: The dendrogram

3.3.4.3 Comparison

Agglomerative hierarchical clustering and divisive hierarchical clustering are two different methods for creating hierarchical clusters from data. Here

are some key differences between the two methods:



*Figure 3.2.4.2: Agglomerative hierarchical clustering
vs divisive hierarchical clustering*

- **Direction:** Agglomerative hierarchical clustering is a bottom-up approach, where the algorithm starts with each data point as its own cluster and successively merges the closest pairs of clusters until all points belong to a single cluster. Divisive hierarchical clustering, on the other hand, is a top-down approach, where the algorithm starts with all data points in a single cluster and successively splits the cluster into smaller ones until each point is in its own cluster.

-
- **Complexity:** Agglomerative clustering is computationally simpler than divisive clustering, as it involves fewer decisions and typically results in fewer clusters. Divisive clustering, on the other hand, involves more decision-making and can result in a larger number of clusters.
 - **Interpretation:** Agglomerative clustering tends to produce clusters that are more natural and interpretable, as the algorithm merges clusters based on their similarity. In divisive clustering, the clusters may not be as natural, as the algorithm splits clusters based on their dissimilarity.
 - **Flexibility:** Agglomerative clustering is more flexible than divisive clustering in terms of the distance metric used to calculate the similarity between clusters, as different distance metrics can be used. Divisive clustering, on the other hand, is typically limited to a single distance metric.

- **Scalability:** Agglomerative clustering is generally more scalable than divisive clustering, as the latter involves more decision-making and can be computationally expensive for large datasets.

*Table 3.3.4.1: Agglomerative hierarchical clustering
vs divisive hierarchical clustering*

<i>Aspect</i>	<i>Agglomerative Clustering</i>	<i>Divisive Clustering</i>
<i>Direction</i>	Bottom-up	Top-down
<i>Complexity</i>	Lower	Higher
<i>Interpreta- tion</i>	More natural	Less natu- ral
<i>Flexibility</i>	More flexible	Less flexi- ble
<i>Scalability</i>	More scalable	Less scalable

3.4.3 Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

DBSCAN is a popular unsupervised machine learning algorithm used for clustering spatial data. The algorithm is particularly effective for datasets with high density areas and noise, and can identify clusters of arbitrary shapes.



Spatial data can be represented using different types of data structures, such as points, lines, polygons, or raster grids, depending on the nature of the data and the level of detail required. Examples of spatial data include maps, satellite images, aerial photographs, GPS data, weather data, population data, and business data with location information. For example, spatial clustering can be used to identify clusters of crime incidents in a city, clusters of disease outbreaks in a region, or clusters of customer locations for a retail store.

DBSCAN works by grouping together points that are close to each other in a dense region of the dataset. It does this by defining two parameters:

epsilon (ϵ), which specifies the radius of a neighborhood around each data point, and minimum points (MinPts), which specifies the minimum number of points required to form a dense region.

The algorithm proceeds as follows:

1. Randomly select an unvisited point in the dataset.
2. If the point has at least MinPts neighbors within distance ϵ , it is considered a core point and a new cluster is started.
3. If the point does not have enough neighbors to be a core point, but it is within ϵ distance of another core point, it is added to that cluster.
4. If the point is not a core point and is not within ϵ distance of any core points, it is considered noise and is not added to any cluster.

-
5. Repeat steps 1-4 for all unvisited points in the dataset.

The resulting clusters are formed by connecting the core points and their neighbouring points. The algorithm can also identify noise points, which do not belong to any cluster.

3.3.3.1 Example to implements the DBSCAN algorithm

Suppose we have a dataset of 10 points in a two-dimensional space:

(1, 2), (1, 1), (2, 1), (2, 2), (8, 8), (8, 9), (9, 8), (9, 9),
(15, 15), (15, 16)

We want to cluster these points using DBSCAN with $\epsilon = 2$ and $\text{MinPts} = 3$.

The algorithm proceeds as follows:

1. Randomly select an unvisited point in the dataset. Let's start with (1, 2).
2. Check its neighborhood within $\epsilon = 2$ distance. It has 3 neighbors: (1, 1), (2, 1), and

(2, 2). Since it has at least $\text{MinPts} = 3$ neighbors, it is considered a core point and a new cluster is started with (1, 2), (1, 1), (2, 1), and (2, 2).

3. Now we move on to the next unvisited point, which is (8, 8). It has 3 neighbors: (8, 9), (9, 8), and (9, 9), so it is considered a core point and a new cluster is started with (8, 8), (8, 9), (9, 8), and (9, 9).
4. The next unvisited point is (15, 15). It has no neighbors within $\epsilon = 2$ distance, so it is considered noise and not added to any cluster.
5. The remaining points (1, 1), (2, 1), (2, 2), (15, 16) are not core points, but they are within $\epsilon = 2$ distance of core points. Thus, they are added to the two existing clusters: (1, 1), (2, 1), and (2, 2) are added to the first cluster, and (15, 16) is added to the noise cluster.

-
6. The algorithm finishes because all points have been visited.

After clustering, the dataset is divided into two clusters, one containing (1, 2), (1, 1), (2, 1), and (2, 2), and the other containing (8, 8), (8, 9), (9, 8), and (9, 9). The point (15, 16) is considered noise and not assigned to any cluster.

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.cluster import DBSCAN

# Define dataset

data = np.array([(1, 2), (1, 1), (2, 1), (2, 2), (8,
8), (8, 9), (9, 8), (9, 9), (15, 15), (15, 16)])

# Define DBSCAN model

dbscan = DBSCAN(eps=2, min_samples=3)

# Fit model and predict clusters

clusters = dbscan.fit_predict(data)

# Plot data points and clusters
```

```
colors = ['blue', 'green', 'orange', 'purple',
'red', 'cyan', 'magenta', 'yellow']

for i in range(len(set(clusters))):

    color = colors[i % len(colors)]

    plt.scatter(data[clusters == i][:, 0], data[
clusters == i][:, 1], color=color)

# Set plot title and axis labels

plt.title('DBSCAN Clustering')

plt.xlabel('X')

plt.ylabel('Y')


# Display plot

plt.show()
```

The Python code above implements the DBSCAN algorithm to cluster a set of 2D points. First, the dataset of 10 points is defined as a list of tuples. The **sklearn** library is then imported, as it provides an implementation of the DBSCAN algorithm.

The **DBSCAN** object is initialized with **eps=2** and **min_samples=3**, which specifies the maximum distance between points in a cluster and the minimum number of points required to form a cluster.

The **fit_predict()** method is then called on the **DBSCAN** object, passing in the dataset. This method fits the DBSCAN model to the data and returns the cluster labels for each point. The clusters are plotted using **matplotlib**. The **scatter()** method is used to plot each point in the dataset, with a different color assigned to each cluster label. The resulting plot clearly shows the two clusters and the noise point.

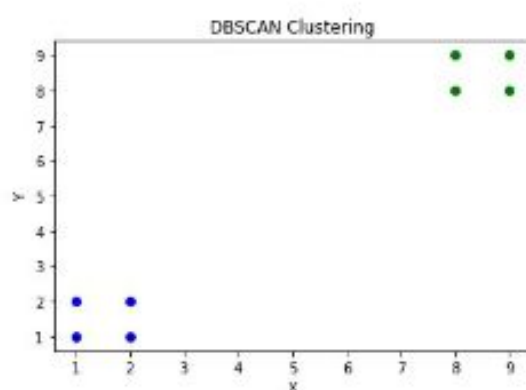


Figure 3.2.1: Output illustration of DBSCAN

3.4.4 Gaussian Mixture Models (GMM)

Gaussian Mixture Models (GMM) is a clustering algorithm that is used to model probability distributions of multi-dimensional data. The idea behind GMM is that the data points are generated from a mixture of Gaussian distributions, where each distribution represents a cluster. Therefore, the goal of GMM is to find the parameters of these Gaussian distributions, namely the mean and the covariance matrix, that best explain the data.

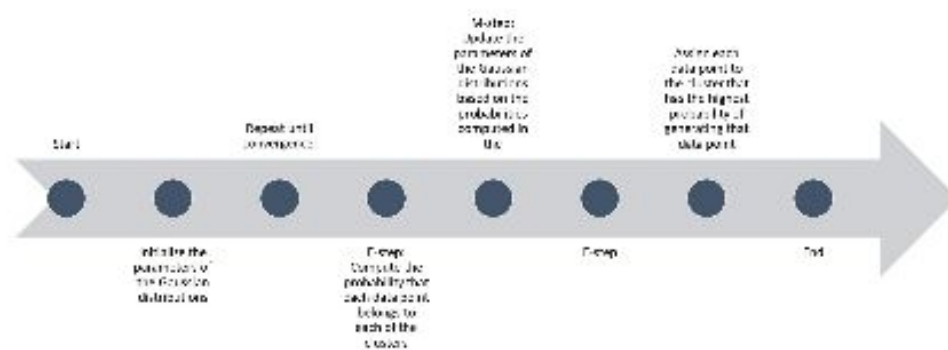


Figure 3.2.1: 3.3: Steps in Gaussian Mixture Models (GMM)

The GMM algorithm works as follows:

1. Initialization: Start by randomly initializing the parameters of the Gaussian distributions, including the mean and the covariance matrix. Also, we need to specify the

number of clusters we want to fit in the data.

2. Expectation step (E-step): In this step, we compute the probability that each data point belongs to each of the clusters. This is done by computing the conditional probability of each data point given the parameters of the Gaussian distributions. Specifically, we calculate the probability of observing each data point given the mean and covariance of each Gaussian distribution.
3. Maximization step (M-step): In this step, we update the parameters of the Gaussian distributions based on the probabilities computed in the E-step. Specifically, we update the mean and covariance matrix of each Gaussian distribution by taking a weighted average of the data points, where the weights are given by the probabilities computed in the E-step.

-
4. Repeat E-step and M-step until convergence: We iterate the E-step and M-step until the algorithm converges, that is, until the log-likelihood of the data stops increasing significantly.
 5. Cluster assignment: After convergence, we assign each data point to the cluster that has the highest probability of generating that data point.

3.4.5 Spectral clustering

Spectral clustering is a data partitioning algorithm used in machine learning and artificial intelligence. It uses the connectivity between data points to form clusters. Spectral clustering uses eigenvalues and eigenvectors of the data matrix to forecast the data into lower dimensions space to cluster the data points. The eigenvalues correspond to the different frequencies in the data, and by looking at the distribution of these eigenvalues, you can get an idea of how many clusters are present.

3.3.5.1 Spectral clustering has the following steps:

1. Uses eigenvalues and eigenvectors of the data matrix to forecast the data into lower dimensions space
2. Constructs a similarity matrix using the eigenvalues and their associated eigenvectors
3. Uses the similarity matrix to group the data points into clusters

Dimensionality reduction algorithms

Dimensionality reduction algorithms are techniques used in machine learning and data analysis to reduce the number of features or variables in a dataset. These algorithms aim to retain the essential information while minimizing the loss of data quality. Here are some commonly used dimensionality reduction algorithms

In many real-world applications, datasets can have a large number of features or variables. High-

dimensional data can pose challenges such as increased computational complexity, increased risk of overfitting, and difficulties in visualizing and interpreting the data. Dimensionality reduction techniques are employed to address these challenges by reducing the number of features while retaining the crucial information within the data.



High-dimensional data refers to datasets with a large number of features, where each feature represents a variable or attribute. In a dataset with images, each pixel may be treated as a feature, resulting in high-dimensional data.

The primary goal is to preserve the essential patterns, structures, and relationships present in the original data. Dimensionality reduction aims to minimize the loss of relevant information while reducing the dataset's dimensionality

3.4.1 Applications

-
- **Image Processing:** In image datasets, reducing the dimensionality can be crucial for tasks like facial recognition or object detection.
 - **Genomics:** Genomic datasets with numerous genes may benefit from dimensionality reduction for improved analysis and interpretation.
 - **Text Data:** Text datasets with a large number of features (words) can be streamlined for more efficient natural language processing.

3.4.2 Popular types of Dimensionality reduction algorithms

- **Linear Techniques (e.g., PCA):** Focus on linear combinations of features to reduce dimensionality.
- **Non-linear Techniques (e.g., t-SNE):** Capture complex relationships and structures in the data.

- **Autoencoders:** Neural network-based approaches that learn hierarchical representations of data.

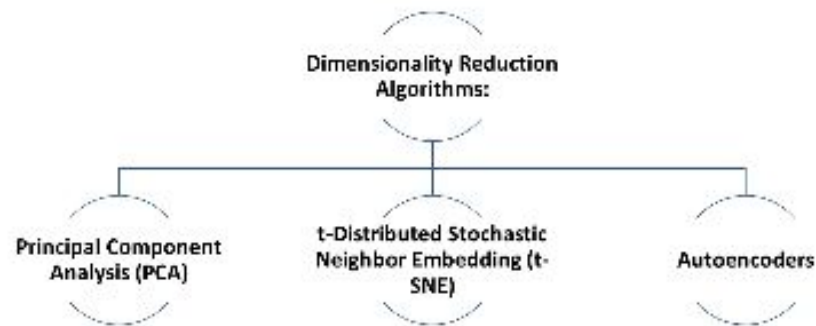


Figure 3.4.1 Dimensionality reduction algorithms

3.5 Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a widely used linear dimensionality reduction technique that transforms high-dimensional data into a new coordinate system, capturing the most significant variance in the data. The new coordinates, called principal components, are orthogonal and ordered by the amount of variance they explain. PCA is particularly useful for visualizing data, reducing

computational complexity, and identifying important patterns in the dataset.

3.5.1 Steps in Principal Component Analysis:

- **Standardization of Data**

Standardize the features to have zero mean and unit variance. This step ensures that all features contribute equally to the analysis.

- **Compute Covariance Matrix**

Calculate the covariance matrix of the standardized data. The covariance matrix provides information about the relationships between different features.

- **Calculate Eigenvectors and Eigenvalues**

Find the eigenvectors and eigenvalues of the covariance matrix. Eigenvectors represent the directions of maximum variance, and eigenvalues indicate the magnitude of variance along these directions.

- **Sort Eigenvectors by Eigenvalues**

Sort the eigenvectors in descending order based on their corresponding eigenvalues.

The eigenvector with the highest eigenvalue corresponds to the principal component with the most significant variance.

- **Select Principal Components**

Choose the top 'k' eigenvectors to form a matrix (projection matrix) where each column is a principal component. The value of 'k' is determined by the desired dimensionality of the reduced dataset.

- **Transform Data**

Multiply the original standardized data by the projection matrix to obtain the new coordinates in the principal component space. This transformation reduces the dimensionality of the data.



In PCA, eigenvectors represent the directions of maximum variance in the data, and eigenvalues indicate the magnitude of this variance along those directions. The principal components are formed by the eigenvectors, and the eigenvalues help determine the importance of each principal component. The

eigenvectors are ordered based on their corresponding eigenvalues, with higher eigenvalues indicating more significant variance.

3.5.2 Example of Principal Component Analysis (PCA) using Python with the popular scikit-learn library

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.decomposition import PCA
```

```
from sklearn.datasets import load_iris
```

```
# Load the iris dataset
```

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
# Standardize the features
```

```
X_std = (X - np.mean(X, axis=0)) / np.std(X,  
axis=0)
```

```
# Apply PCA
```

```
pca = PCA(n_components=2)
```

```
X_pca = pca.fit_transform(X_std)
```

```
# Visualize the results
```

```
plt.figure(figsize=(8, 6))
```

```
for i in range(len(set(y))):
```

```
    plt.scatter(X_pca[y == i, 0], X_pca[y == i, 1],  
label=f'Class {i}')
```

```
plt.title('PCA of Iris Dataset')
```

```
plt.xlabel('Principal Component 1')
```

```
plt.ylabel('Principal Component 2')
```

```
plt.legend()
```

```
plt.show()
```

This example uses the Iris dataset, which is a popular dataset for demonstrating PCA. The code performs the following steps:

- **Load the Iris dataset:** The dataset is loaded using scikit-learn's **load_iris** function.
- **Standardize the features:** Standardization is applied to ensure that all features have zero mean and unit variance.
- **Apply PCA:** The **PCA** class from scikit-learn is used to perform PCA. In this case, we specify **n_components=2** to reduce the data to two dimensions.
- **Visualize the results:** The transformed data is plotted in a 2D scatter plot, with different colors representing different classes.

Make sure to install the necessary libraries before running the code. You can install scikit-learn and matplotlib using

```
pip install scikit-learn matplotlib
```

The aim of the provided Python code is to demonstrate the application of Principal Component Analysis (PCA) on the Iris dataset. PCA is a dimensionality reduction technique commonly used in machine learning and data analysis.

3.8 t-Distributed Stochastic Neighbor Embedding (t-SNE):

t-SNE is a nonlinear dimensionality reduction technique commonly used for visualizing high-dimensional data in lower-dimensional spaces. Unlike linear techniques such as PCA, t-SNE is particularly effective at capturing complex structures and maintaining local relationships between data points. Here are the steps involved in the t-SNE algorithm:

- **Define Similarities**

For each pair of data points in the high-dimensional space, compute conditional probabilities that represent their similarities. These probabilities are based on the pairwise similarities in the high-dimensional space and are defined using a Gaussian distribution.



High-dimensional data refers to datasets with a large number of features or dimensions. In the context of machine learning, each feature represents a distinct characteristic or attribute of the data. The term "high-dimensional" becomes particularly relevant when the number of features is significantly larger than the number of observations or instances in the dataset.

- **Symmetrize the Similarities**

Make the conditional probabilities symmetrical by taking the average of P_{ij} and P_{ji} where

P_{ij} represents the probability of picking j as a neighbor if i is picked as a neighbor.

- **Define Similarities in Low-Dimensional Space:**

Define similarities in the low-dimensional space. This involves computing conditional probabilities Q_{ij} that represent similarities between points in the low-dimensional space. These probabilities are defined using a Student's t-distribution.

- **Compute Gradient**

Compute the gradient of the Kullback-Leibler (KL) divergence between the high-dimensional and low-dimensional probability distributions with respect to the coordinates of the data points in the low-dimensional space. The gradient provides information on how to move the points to reduce the divergence.

- **Update Coordinates:**

Update the coordinates of the data points in the low-dimensional space based on the computed gradient. This step involves using an optimization algorithm, typically gradient descent, to minimize the KL divergence.

- **Repeat the Process**

Iterate the process until convergence or for a specified number of iterations. During each iteration, the algorithm refines the positions of the data points in the low-dimensional space to better represent the relationships present in the high-dimensional space.

- **Visualization:** t-SNE is often used for visualizing clusters, patterns, and structures in high-dimensional data. It is particularly popular in applications such as image processing, natural language processing, and biological data analysis, where capturing intricate relationships between data points is crucial for interpretation.

3.8.1 Example of a Python code using the t-SNE algorithm from the scikit-learn library.

This code applies t-SNE to the famous Iris dataset and visualizes the data in a 2D spac.

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn import datasets

from sklearn.manifold import TSNE


# Load the Iris dataset

iris = datasets.load_iris()

X = iris.data

y = iris.target


# Apply t-SNE

tsne = TSNE(n_components=2, random_s-
tate=42)
```

```
X_tsne = tsne.fit_transform(X)

# Visualize the results

plt.figure(figsize=(8, 6))

for i in range(len(set(y))):

    plt.scatter(X_tsne[y == i, 0], X_tsne[y == i,
1], label=f'Class {i}')

plt.title('t-SNE of Iris Dataset')

plt.xlabel('t-SNE Component 1')

plt.ylabel('t-SNE Component 2')

plt.legend()

plt.show()
```

3.6.2 Explanation of the code

Load the Iris dataset: The Iris dataset is loaded using scikit-learn's **datasets** module.

Apply t-SNE: The **TSNE** class from scikit-learn is used to perform t-SNE. We specify **n_components=2** to reduce the data to two dimensions.

Visualize the Results: The transformed data is plotted in a 2D scatter plot, with different colors representing different classes of iris flowers.

This example demonstrates how t-SNE can be applied to reduce the dimensionality of the data and visualize it in a lower-dimensional space. It helps in observing patterns and relationships between instances, especially in cases where complex structures exist in the data.

3.9 Autoencoders

Autoencoders are a type of artificial neural network used for unsupervised learning and dimensionality reduction. The primary objective of an autoencoder is to learn an efficient representation of input data by encoding it into a lower-dimensional space and then decoding it back to the

original form. Here are the steps involved in the autoencoder algorithm:

3.7.1 Architecture Setup

An autoencoder consists of an encoder and a decoder. The encoder transforms the input data into a compressed representation, and the decoder reconstructs the input from this representation. Both the encoder and decoder are neural networks.

Train the encoder to map the input data to a lower-dimensional representation. This involves feeding the input data through the encoder and comparing the output with the original input. The objective is to minimize the reconstruction error, typically measured using a loss function like Mean Squared Error (MSE).

Train the decoder to reconstruct the input data from the compressed representation generated by the encoder. This involves feeding the encoded representation through the decoder and comparing the output with the original input. The same

loss function is used to measure the reconstruction error.

The overall training objective is to minimize the sum of the reconstruction errors from both the encoder and decoder. The network adjusts its parameters (weights and biases) through backpropagation and optimization algorithms like stochastic gradient descent to achieve this minimization.

The layer in the middle of the autoencoder, where the data is compressed, is referred to as the bottleneck layer or latent space. The size of this layer determines the dimensionality of the compressed representation.

Autoencoders are trained without explicit labels. The network learns to encode and decode the input data based solely on the patterns present in the data itself. This makes autoencoders well-suited for unsupervised learning tasks, such as feature learning and dimensionality reduction.

3.7.2 Example of a simple autoencoder implemented in Python using the Keras library

```
import numpy as np

import matplotlib.pyplot as plt

from tensorflow.keras.layers import Input,
Dense

from tensorflow.keras.models import Model

from tensorflow.keras.datasets import mnist

# Load the MNIST dataset

(x_train, _), (x_test, _) = mnist.load_data()

# Normalize pixel values between 0 and 1
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# Flatten the images
```

```
x_train = x_train.reshape((len(x_train),  
np.prod(x_train.shape[1:])))
```

```
x_test = x_test.reshape((len(x_test),  
np.prod(x_test.shape[1:])))
```

```
# Define the autoencoder architecture
```

```
input_img = Input(shape=(784,))
```

```
encoded = Dense(128, activation='relu')(in-  
put_img) # Encoder layer
```

```
decoded = Dense(784, activation='sigmoid')  
(encoded) # Decoder layer
```

```
# Build the autoencoder model
```

```
autoencoder = Model(input_img, decoded)
```

```
# Compile the model
```

```
autoencoder.compile(optimizer='adam',  
loss='binary_crossentropy')
```

```
# Train the autoencoder
```

```
autoencoder.fit(x_train, x_train, epochs=50,  
batch_size=256, shuffle=True, validation_  
data=(x_test, x_test))
```

```
# Encode and decode some digits
```

```
encoded_imgs = autoencoder.predict(x_test)
```

```
# Plot the original and reconstructed digits
```

```
n = 10 # Number of digits to display
```

```
plt.figure(figsize=(20, 4))
```

```
for i in range(n):
```

```
    # Display original
```

```
    ax = plt.subplot(2, n, i + 1)
```

```
    plt.imshow(x_test[i].reshape(28, 28))
```

```
    plt.gray()
```

```
    ax.get_xaxis().set_visible(False)
```

```
ax.get_yaxis().set_visible(False)

# Display reconstruction
ax = plt.subplot(2, n, i + 1 + n)

plt.imshow(encoded_imgs[i].reshape(28,
28))

plt.gray()

ax.get_xaxis().set_visible(False)
ax.get_yaxis().set_visible(False)

plt.show()
```

This is an example of a simple autoencoder implemented in Python using the Keras library, which is a high-level neural networks API running on top of TensorFlow. This example uses the popular MNIST dataset for simplicity

This code defines a simple autoencoder with one hidden layer and trains it on the MNIST dataset. It

then visualizes the original and reconstructed images.

```
pip install tensorflow matplotlib
```

Make sure to install TensorFlow and matplotlib before running the code

CHAPTER 4

SEMI-SUPERVISED LEARNING

4.1 Introduction

Semi-supervised learning classification is a type of machine learning approach in which a model is trained on a combination of labeled and unlabeled data. In traditional supervised learning, the model is trained on a labeled dataset, meaning that each data point is accompanied by a corresponding label or target output. However, labeled datasets can be expensive and time-consuming to create, and may not always be available in certain domains.

In semi-supervised learning classification, the model is trained on a smaller set of labeled data,

supplemented by a larger set of unlabeled data. The unlabeled data may provide additional information about the underlying structure of the data and can be used to improve the accuracy of the model.

One common approach to semi-supervised learning classification is to use a technique called "self-training." In self-training, the model is initially trained on the labeled data, and then used to predict labels for the unlabeled data. These predicted labels are then added to the labeled dataset, and the model is retrained on the expanded labeled dataset. This process can be repeated iteratively to improve the performance of the model.

Another approach to semi-supervised learning classification is to use a technique called "co-training." In co-training, two different models are trained on different subsets of the feature space, with each model using a different subset of the labeled data. The models then exchange information about their predictions on the unlabeled data,

and this information is used to improve the accuracy of both models.

There are several types of semi-supervised learning, each with its own unique approach to incorporating unlabeled data into the learning process. Here are some of the most common types of semi-supervised learning:

- **Self-training:** In self-training, a model is first trained on a small labeled dataset, and then used to make predictions on the unlabeled data. The most confident predictions are then added to the labeled dataset, and the model is retrained on the expanded dataset. This process is repeated iteratively until the model reaches a certain level of performance.
- **Co-training:** In co-training, two or more models are trained on different subsets of the feature space. Each model is trained on a subset of the labeled data, and then used to make predictions on the unlabeled data.

The predictions are then used to augment the training data for the other model, and vice versa. This process continues until the models converge.

- **Multi-view learning:** In multi-view learning, multiple views or representations of the data are used to learn a single model. Each view may correspond to a different feature space or modality, and the model must learn to combine information from all views to make predictions.
- **Graph-based learning:** In graph-based learning, the data is represented as a graph, where nodes correspond to data points and edges represent relationships between them. The labeled data is used to construct a graph, and then the graph is used to propagate labels to the unlabeled data.
- **Semi-supervised clustering:** In semi-supervised clustering, the goal is to partition the data into groups or clusters, but only a small

subset of the data is labeled. The labeled data is used to guide the clustering process, and the resulting clusters can then be used for classification or other tasks.

4.2 The Self-Training Algorithm

The Self-Training Algorithm is a popular approach for semi-supervised learning that involves iteratively training a model on a small labeled dataset and then using it to predict labels for the unlabeled data. The most confident predictions are then added to the labeled dataset, and the model is retrained on the expanded dataset. This process is repeated until the model reaches a certain level of performance.

Here are the general steps of the Self-Training Algorithm:

1. Train a model on a small labeled dataset:
Start by training a model on a small labeled dataset, using a supervised learning algo-

rithm such as logistic regression or support vector machines. This initial model will serve as a starting point for the iterative process.

2. Predict labels for the unlabeled data: Use the trained model to make predictions on the unlabeled data. The predictions will provide a tentative label for each unlabeled instance.
3. Select the most confident predictions: Select the unlabeled instances with the highest predicted probabilities or scores for their respective classes. These instances will be added to the labeled dataset.
4. Add the most confident predictions to the labeled dataset: Add the selected instances to the labeled dataset, along with their predicted labels.
5. Retrain the model on the expanded dataset: Retrain the model on the expanded labeled

dataset, including the newly labeled instances. This updated model will be used to predict labels for the remaining unlabeled data.

6. Repeat until convergence: Repeat steps 2-5 until the performance of the model on a validation set stops improving, or until a predetermined number of iterations is reached.

The key idea behind the Self-Training Algorithm is that by incorporating the most confident predictions into the labeled dataset, the model can leverage the additional information to improve its performance. However, there are some limitations to this approach, such as the risk of propagating errors and the potential for the model to become overconfident. Therefore, it is important to carefully monitor the performance of the model and to use appropriate techniques to mitigate these risks.

Example of Text classification tasks

One example of a real-world application of semi-supervised learning is in the field of natural language processing, specifically in text classification tasks. Here's an example case study

Problem: A company has a large volume of customer feedback data in the form of text reviews, but only a small portion of it has been manually labeled with sentiment labels (positive, negative, neutral). The company wants to build a sentiment classification model to automatically classify new customer reviews as positive, negative, or neutral.

Approach: The company can use a semi-supervised learning approach to leverage the large volume of unlabeled data to improve the performance of the sentiment classification model. Here's how:

- Train a sentiment classification model on the small labeled dataset: The company can start by training a sentiment classification model on the small labeled dataset using a supervised learning algorithm such

as logistic regression or support vector machines.

- Use the trained model to predict labels for the unlabeled data: The trained model can be used to predict the sentiment labels for the large volume of unlabeled data.
- Select the most confident predictions: The company can select the instances with the highest predicted probabilities or scores for their respective classes. These instances will be added to the labeled dataset.
- Add the most confident predictions to the labeled dataset: The company can manually label the selected instances with the predicted labels, and add them to the labeled dataset.
- Retrain the model on the expanded dataset: The model can be retrained on the expanded labeled dataset, including the newly labeled instances.

-
- Evaluate the performance on a validation set: The performance of the updated model can be evaluated on a validation set to see if it has improved.
 - Repeat the process until convergence: The company can repeat steps 2-6 until the performance of the model on the validation set stops improving or reaches a satisfactory level.

Using this approach, the company can leverage the large volume of unlabeled data to improve the performance of the sentiment classification model. By selecting the most confident predictions and manually labeling them, the company can gradually increase the size of the labeled dataset and improve the accuracy of the model. This approach can be particularly useful in cases where manual labeling is expensive or time-consuming.

```
# Import the required libraries
```

```
import pandas as pd
```

```
from sklearn.feature_extraction.text import  
CountVectorizer
```

```
from    sklearn.model_selection    import  
train_test_split
```

```
from sklearn.semi_supervised import Self-  
TrainingClassifier
```

```
from sklearn.linear_model import Logisti-  
cRegression
```

```
# Load the dataset
```

```
df = pd.read_csv('customer_reviews.csv')
```

```
# Split the dataset into labeled and unlabeled  
data
```

```
X_labeled, X_unlabeled, y_labeled, y_unla-  
beled = train_test_split(
```

```
    df['review'], df['sentiment'], test_size=0.5,  
    stratify=df['sentiment'])
```

Preprocess the text data using CountVectorizer

vectorizer = CountVectorizer(stop_words='english')

X_labeled = vectorizer.fit_transform(X_labeled)

X_unlabeled = vectorizer.transform(X_unlabeled)

Train a logistic regression model on the labeled data

classifier = LogisticRegression()

classifier.fit(X_labeled, y_labeled)

Use Self-Training Algorithm to iteratively label the unlabeled data

self_training_classifier = SelfTrainingClassifier(classifier)

```
self_training_classifier.fit(X_labeled, y_la-  
beled)
```

```
while True:
```

```
    # Predict the labels for the unlabeled data
```

```
    predicted_labels = self_training_classifi-  
er.predict(X_unlabeled)
```

```
    # Select the most confident predictions and  
add them to the labeled data
```

```
    confidences = self_training_classifier.pre-  
dict_proba(X_unlabeled).max(axis=1)
```

```
    most_confident_indices = confi-  
dences.argsort()[::-1][:100]
```

```
    X_labeled = pd.concat([X_labeled, X_unla-  
beled.iloc[most_confident_indices]])
```

```
    y_labeled = pd.concat([y_labeled, pd.Se-  
ries(predicted_labels[most_confiden-  
t_indices]])])
```

Remove the most confident predictions from the unlabeled data

X_unlabeled = X_unlabeled.drop(X_unlabeled.index[most_confident_indices])

Retrain the classifier on the updated labeled data

self_training_classifier.fit(X_labeled, y_labeled)

Check if the performance on the validation set has stopped improving

if **self_training_classifier.score**(vectorizer.transform(df['review']), df['sentiment']) >= 0.85:

break

Evaluate the performance of the final model

X_test = vectorizer.transform(['This product is great!', 'This product is terrible!'])

```
y_test    =    self_training_classifier.predic-  
t(X_test)  
  
print(y_test)
```

- In this first block of code, we import the required libraries. Pandas is used for data manipulation and analysis, CountVectorizer is used to preprocess the text data, train_test_split is used to split the dataset into labeled and unlabeled data, SelfTrainingClassifier is used to implement the Self-Training Algorithm, and LogisticRegression is used as the base classifier.
- Next, we load the customer reviews dataset using pandas. The dataset should be a CSV file with columns for the reviews and their corresponding sentiment labels.
- We split the dataset into labeled and unlabeled data using the train_test_split function. We use 50% of the data as unlabeled

data, and stratify the split based on the sentiment labels to ensure that the distribution of labels is similar in the labeled and unlabeled data.

- We preprocess the text data using `CountVectorizer`, which converts the reviews into a matrix of word counts. We remove common English stop words to avoid including them as features.
- We train a logistic regression model on the labeled data as the base classifier for the Self-Training Algorithm.
- We create a `SelfTrainingClassifier` object with the logistic regression model as the base classifier, and fit it on the labeled data.
- We use a while loop to iteratively label the unlabeled data using the Self-Training Algorithm. In each iteration, we predict the labels for the unlabeled data using the `SelfTrainingClassifier`'s `predict` method. We

also calculate the confidence of each prediction using the `predict_proba` method, and select the 100 most confident predictions using `argsort` and slicing. We add these predictions to the labeled data and remove them from the unlabeled data. We then refit the `SelfTrainingClassifier` on the updated labeled data. We repeat this process until the performance on the validation set (the entire dataset) reaches a satisfactory level of accuracy, which in this case is 85%.

Co-Training Algorithm

Co-Training is a semi-supervised learning algorithm that is called so because it involves training multiple classifiers simultaneously in a collaborative manner. The algorithm splits the dataset into two or more views, and each view is used to train a separate classifier.

The trained classifiers work together in a coordinated way, exchanging labeled instances to augment each other's training set.



The classifiers are algorithms that take input data and assign a label or class to it based on some learned criteria. The goal of a classifier is to learn a decision boundary that separates different classes in the data space.

This collaborative process is what makes the algorithm "co-" in co-training. The basic assumption behind co-training is that if multiple independent classifiers agree on a prediction for an unlabeled instance, then the prediction is more likely to be accurate, and the instance can be added to the labeled dataset for subsequent rounds of training.

The algorithm works as follows:

1. The dataset is split into two or more views, each containing a different subset of features.
2. The classifiers are initialized with a small amount of labeled data, and the rest of the data is left unlabeled.

-
3. The classifiers are trained independently on their respective views of the data.
 4. The classifiers make predictions on the unlabeled data, and the instances for which both classifiers agree with high confidence are added to the labeled dataset.
 5. The labeled data is then used to retrain the classifiers, and the process is repeated until convergence or until the desired accuracy is achieved.

The key idea behind Co-Training is that if multiple independent classifiers agree on a prediction for an unlabeled instance, then the prediction is more likely to be accurate. By exchanging labeled instances, the classifiers can effectively learn from each other and improve their performance on the unlabeled data.

Initialize Classifiers

Split Dataset into Views

Train Classifiers on Their Respective Views

Make Predictions on Unlabeled Data

Exchange Labeled Instances

Retrain Classifiers on the Combined Labeled Data

Repeat Until Convergence or Desired Accuracy is Achieved

4.3.1 Example : Implementing Co-Training for Semi-Supervised Learning using Multiple Views of Data in Python

```
from sklearn.datasets import make_classification
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.metrics import accuracy_score
```

```
# generate a toy dataset with 1000 samples  
and 10 informative features
```

```
X, y = make_classification(n_samples=1000,
n_classes=2, n_informative=10, random_s-
tate=42)

# split the data into two views (sets of fea-
tures)

X1 = X[:, :5]

X2 = X[:, 5:]

# split the data into labeled and unlabeled
portions

X1_labeled, X1_unlabeled, y_labeled, y_unla-
beled = train_test_split(X1, y, test_size=0.9,
stratify=y, random_state=42)

X2_labeled, X2_unlabeled, _, _ = train_test_s-
plit(X2, y, test_size=0.9, stratify=y, ran-
dom_state=42)

# initialize two classifiers (decision trees)

clf1 = DecisionTreeClassifier(random_s-
tate=42)
```

```
clf2 = DecisionTreeClassifier(random_s-  
tate=42)  
  
# train the classifiers on the labeled data for  
each view  
  
clf1.fit(X1_labeled, y_labeled)  
clf2.fit(X2_labeled, y_labeled)  
  
# iterate until convergence  
for i in range(10):  
    # make predictions on the unlabeled data  
    for each view  
  
        y1_pred = clf1.predict(X1_unlabeled)  
        y2_pred = clf2.predict(X2_unlabeled)  
  
        # select the top-k most confident predic-  
        tions from each classifier  
  
        k = 10  
  
        y1_topk = y1_pred.argsort()[-k:][::-1]  
        y2_topk = y2_pred.argsort()[-k:][::-1]
```

```
# add the selected samples to the labeled
data and retrain the classifiers
```

```
X1_labeled    =    np.vstack((X1_labeled,
X1_unlabeled[y2_topk]))
```

```
X2_labeled    =    np.vstack((X2_labeled,
X2_unlabeled[y1_topk]))
```

```
y_labeled     =    np.hstack((y_labeled,
y1_pred[y1_topk]))
```

```
clf1.fit(X1_labeled, y_labeled)
```

```
clf2.fit(X2_labeled, y_labeled)
```

```
# remove the selected samples from the un-
labeled data
```

```
X1_unlabeled  =  np.delete(X1_unlabeled,
y2_topk, axis=0)
```

```
X2_unlabeled  =  np.delete(X2_unlabeled,
y1_topk, axis=0)
```

```
# evaluate the performance of the classi-
fiers on a test set
```

```
X_test, y_test = make_classification(n_samples=100, n_classes=2, n_informative=10, random_state=42)

y1_pred = clf1.predict(X_test[:, :5])

y2_pred = clf2.predict(X_test[:, 5:])

y_pred = np.mean([y1_pred, y2_pred],
axis=0)

accuracy = accuracy_score(y_test, y_pred >
0.5)

print(f"Iteration {i}: accuracy={accuracy:.2f}")
```

Here , a toy dataset with 1000 samples and 10 informative features is generated using the **make_classification** function from **sklearn.datasets**. The data is split into two views (sets of features) - **X1** with the first 5 features and **X2** with the last 5 features. Then, the data is split into labeled and

accuracy_score from **sklearn.metrics**. Finally, the accuracy is printed for each iteration.

4.4. Multi-view learning

Multi-view learning and semi-supervised learning are two different but complementary techniques in machine learning. Multi-view learning focuses on using multiple representations or feature spaces to improve model performance, while semi-supervised learning aims to leverage both labeled and unlabeled data to improve model performance.

In the context of semi-supervised learning, multi-view learning can be used to improve the quality of the feature representations for both labeled and unlabeled data. By using multiple feature spaces, the model can learn more robust representations that capture different aspects of the data. This can help to reduce the impact of noise and outliers in the data and improve the accuracy of the model.

A common case study example of multi-view learning in semi-supervised learning is in image classification. In this example, the goal is to classify images into different categories, such as "dog" or "cat". However, instead of using just one representation of the image, multiple views of the image are used, such as the raw pixel values, color histogram, and texture features. These different views of the image capture different aspects of the image, such as the overall shape, color distribution, and texture patterns.

The model can then be trained on a combination of labeled and unlabeled images using a semi-supervised learning approach, such as self-training or co-training. The model uses the labeled data to learn how to classify the images, while also using the unlabeled data to improve the quality of the feature representations. By leveraging multiple views of the data, the model can learn more robust representations that are better able to generalize to new, unseen images.

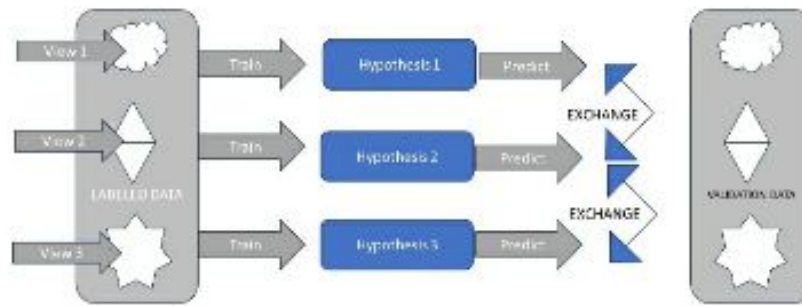


Figure 4.4.1: 4.4.: Multi-view learning

Multi-view learning can be a powerful technique for improving the performance of semi-supervised learning models, particularly in complex and noisy datasets where traditional approaches may struggle to capture all of the relevant information.

4.4.1 Example : Canonical Correlation Analysis (CCA) using the scikit-learn library

```
from sklearn.cross_decomposition import  
CCA
```

```
import numpy as np
```

```
# generate two views of the same data
```

```
X1 = np.random.rand(100, 5) # view 1 has 5  
features
```

```
X2 = np.random.rand(100, 3) # view 2 has 3  
features
```

```
# apply CCA to learn a joint representation
```

```
cca = CCA(n_components=2)
```

```
cca.fit(X1, X2)
```

```
# project the data into the joint space
```

```
X1_joint, X2_joint = cca.transform(X1, X2)
```

```
# use the joint representation to make predic-  
tions
```

```
# (e.g., in a supervised learning task)
```

```
y = np.random.randint(2, size=100) # binary  
classification task
```

```
X_joint = np.hstack((X1_joint, X2_joint))
```

```
clf = LogisticRegression()
```

```
clf.fit(X_joint, y)
```

In this example, we first generate two views of the same data: **X1** has 5 features and **X2** has 3 features. We then apply CCA to learn a joint representation of the two views, using **n_components=2** to project the data into a 2-dimensional space. We then use the joint representation to make predictions on a binary classification task, using logistic regression as the classifier.

4.5 Graph-based learning

Graph-based learning is a type of machine learning algorithm that uses graph structures to represent data and relationships between data points. In this approach, data is represented as nodes in a graph, and relationships between data points are represented as edges. Graph-based learning can be used for a wide range of tasks, including clustering, classification, and anomaly detection.

An example of graph-based learning is spectral clustering. In spectral clustering, a graph is constructed from the data points, where each point is a node in the graph, and edges are added between

nodes that are similar. The graph is then transformed using spectral decomposition, and the resulting eigenvalues and eigenvectors are used to cluster the data.

Graph structures can be used to represent data and relationships between data points in a variety of ways. In general, a graph is a collection of nodes (also known as vertices) and edges that connect those nodes.

In the context of representing data, nodes can represent data points, and edges can represent relationships between those data points. For example, suppose we have a dataset of social media users and their connections to each other. We could represent this data using a graph, where each node represents a user and each edge represents a connection between two users. The edges could be weighted to represent the strength of the connection between the two users.

Graph structures can also be used to represent more complex relationships between data points.

For example, suppose we have a dataset of images of objects and we want to group together images of objects that are visually similar. We could represent each image as a node in a graph, and connect images with edges if they are visually similar (e.g., if they share certain features like color or shape). We could then use graph algorithms to cluster similar images together.

Some advantages of using graph-based learning include:

- **Flexibility:** Graphs can be used to represent a wide range of data types and relationships between data points.
- **Interpretability:** Graphs can be visualized and interpreted in a way that is intuitive for humans.
- **Robustness:** Graph-based algorithms can often handle noisy or incomplete data better than other machine learning approaches.

However, there are also some potential disadvantages to using graph-based learning:

- Scalability: Graph-based algorithms can become computationally expensive for very large datasets.
- Choice of graph structure: There may be multiple ways to represent the same data as a graph, and the choice of graph structure can affect the performance of the algorithm.
- Selection of graph algorithms: There are many graph algorithms to choose from, and selecting the right algorithm for a given task can be challenging.

```
%%html
```

```
<style>
```

```
.graph-container {
```

```
display: flex;
```

```
    justify-content: center;
    flex-wrap: wrap;
}

.graph {
    margin: 10px;
    border: 1px solid black;
}

</style>
```

```
%%python

import networkx as nx

import matplotlib.pyplot as plt

# create an example graph

G = nx.Graph()

G.add_nodes_from([1, 2, 3, 4])
```

```
G.add_edges_from([(1, 2), (1, 3), (2, 3), (3, 4)])

# draw the graph using NetworkX and Mat-
plotlib

pos = nx.spring_layout(G)

fig, ax = plt.subplots(figsize=(5, 5))

nx.draw(G, pos, ax=ax, with_labels=True,
font_weight='bold')

ax.set_title("Example Graph")


# display the graph in an HTML container

from IPython.display import HTML

html_str = f"<div class='graph-
container'>{plt.gcf().canvas.html()}</div>"

HTML(html_str)
```

4.6 Machine learning applications and Uses cases

Machine learning is a transformative field within artificial intelligence that empowers computers to learn patterns and make decisions without explicit programming. It encompasses various types of learning, including supervised learning, unsupervised learning, and semi-supervised learning, each serving distinct purposes in solving real-world problems.

In supervised learning, the algorithm is trained on labeled data, where the input is paired with the corresponding output. This enables the algorithm to learn the mapping between inputs and outputs, making it capable of making predictions or classifications on new, unseen data. Classification and regression are two common tasks in supervised learning.

Classification involves assigning predefined labels to input data. For instance, email classification as spam or not spam, image classification to identify objects like cats or dogs, and customer churn prediction to categorize clients as likely to churn or

not, are classic examples. These applications find relevance in various domains, from filtering unwanted emails to predicting customer behavior.

Regression, on the other hand, deals with predicting a continuous value. Predicting house prices, estimating daily store footfall, and forecasting customer lifetime value are instances where regression models prove valuable. These applications guide decisions in real estate, retail, and customer relationship management.

In recommendation systems, supervised learning is employed to analyze user preferences and suggest items of interest. This is widely seen in platforms like Netflix, where users are recommended movies or TV shows based on their viewing history and preferences, enhancing user experience and content consumption.

Unsupervised learning, in contrast, deals with unlabeled data, aiming to discover inherent patterns or structures within the data. Clustering

and anomaly detection are common unsupervised learning tasks.

Clustering involves grouping similar data points together. Businesses use this approach to segment customers based on purchase history, tailoring marketing strategies for different customer segments. Similarly, grouping documents into topics based on content aids in organizing and understanding large textual datasets.

Anomaly detection identifies unusual patterns or outliers in data. It finds applications in diverse fields such as fraud detection in financial transactions, network intrusion detection in cybersecurity, and identifying rare medical conditions through abnormal patient data.

Semi-supervised learning combines labeled and unlabeled data for training. This approach is beneficial when obtaining labeled data is costly or time-consuming.

In image classification, semi-supervised learning can be employed to classify images using a small amount of labeled data and a large amount of unlabeled data. Similarly, in text classification, it assists in categorizing documents with minimal labeled data, leveraging the vast amounts of available unlabeled text data.

Spam filtering is another domain where semi-supervised learning proves effective. By using a limited set of labeled spam and non-spam emails along with a large set of unlabeled emails, the algorithm learns to distinguish between spam and legitimate messages.

Real-world applications of machine learning abound. For instance, Netflix utilizes machine learning to recommend content based on user preferences, Amazon employs it for product recommendations, Google leverages machine learning to enhance search engine results, and financial institutions rely on it for fraud detection and credit card approval.

Machine learning algorithms have become integral tools in solving complex problems across diverse domains. Whether it's improving user experience through personalized recommendations, enhancing search engine capabilities, or safeguarding financial transactions, the applications of machine learning continue to evolve, making it a driving force in the age of artificial intelligence.